



AKTU

B.Tech III-Sem / MCA



CS IT & CS Allied

COA Computer Organization & Architecture

Crash Course



Unit-2 in One Shot



By Pragya Ma'am

Computer organization and architecture

UNIT 2

Arithmetic and logic unit

Today's Target

- Decimal to binary conversion
- Binary addition
- Booth's algorithm
- AKTU PYQ's

By PRAGYA RAJVANSHI

B.Tech, M.Tech(C.S.E.)

Decimal to binary conversion

How to represent negative number in binary

(-7)

$$\begin{array}{r}
 \begin{array}{cccc}
 8 & 4 & 2 & 1 \\
 0 & 1 & 1 & 1
 \end{array} \\
 +7 \longrightarrow \\
 \begin{array}{r}
 1000 \rightarrow \text{1's complement} \\
 +1 \\
 \hline
 1001 \text{ --- 2's complement}
 \end{array} \\
 (-7)
 \end{array}$$

$$\begin{array}{r}
 8 \ 4 \ 2 \ 1 \\
 1001 \\
 \hline
 -8 + 1 = -7
 \end{array}$$

(-8)

$$\begin{array}{r}
 \begin{array}{cccc}
 16 & 8 & 4 & 2 & 1 \\
 0 & 1 & 0 & 0 & 0 \\
 & 1 & 1 & 1 & 1 \\
 10111 \rightarrow \text{1's complement} \\
 +1 \quad \text{2's complement} \\
 \hline
 11000 \text{ --- } -8
 \end{array} \\
 +8 \longrightarrow
 \end{array}$$

$$\begin{array}{r}
 16 \ 8 \ 4 \ 2 \ 1 \\
 11000 \\
 -16 + 8 = -8
 \end{array}$$

How to add two binary number

$0+1 = 1$
 $1+0 = 1$
 $0+0 = 0$
 $1+1 = 0$ with carry 1
 $1+1+1 = 1$ with carry 1

$$\begin{array}{r} 8 \\ + 7 \\ \hline 15 \end{array}$$

8	4	2	1		
1	0	0	0	-	8
+	0	1	1	-	-
1	1	1	1		

1's complement

$0 \rightarrow 1$ Conversion
 $1 \rightarrow 0$

2's complement

1's complement plus 1

Booth's Algorithm

$$7 * 3 = 21$$

$$\checkmark \quad \underline{111} * \underline{011} = \underline{10101}$$

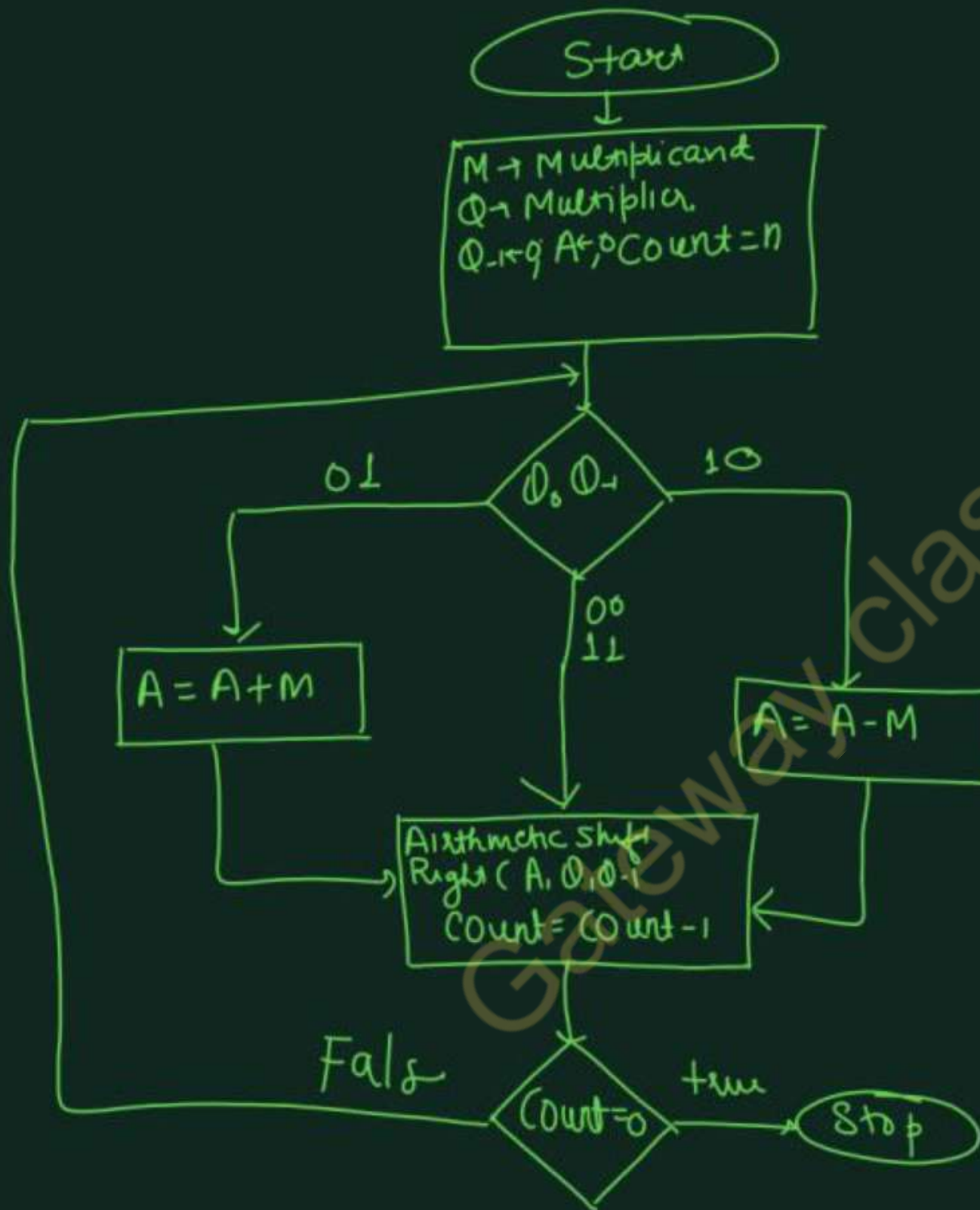
$$\begin{array}{r} 108421 \\ 10101 \end{array}$$

$$\begin{array}{l} -7 * -3 = 21 \\ +7 * +3 = 21 \\ -7 * 3 = -21 \\ 7 * -3 = -21 \end{array}$$

Cases

It is an algorithm used to Multiply two signed number

Q_3, Q_2, Q_1, Q_0
 0 1 1 1



$7 * 3$
 ↑

M = Multiplicand Register
 Q = Multiplier Register
 Q_{-1} → 1 bit Special Register
 A = Accumulator
 Count = no of bit (n)

Case 1 When both numbers are positive

$$\begin{array}{r} 7 * 3 \\ 8421 \\ M=7 \quad \boxed{0111-M} \\ Q=3 \quad \boxed{0011} \\ \hline A = \boxed{0000} \\ Q_{-1} = \boxed{0} \\ \text{Count} = \boxed{4} \end{array}$$

$$\begin{array}{r} A-M \quad -M = (-7) \\ A+(-M) \quad +7 = 0111 \\ \text{1's complement} \quad 1000 \\ \text{2's complement} \quad +1 \\ \hline \quad \quad \quad 1001 \end{array}$$

$$\boxed{-M = 1001}$$

Multiply two numbers (7*3) using Booth's algorithm

A	Q Q_3, Q_2, Q_1, Q_0	Q_{-1}	Operation	Count
Step 1 0000	0011	0	$Q_3Q_{-1} = 10$ $A = A - M$ $A + (-M)$ $0000 + 1001$ $\begin{array}{r} 0000 \\ 1001 \\ \hline 1001 \end{array}$ $(ASR(A, Q_3, Q_{-1}))$	4
		0 1	Lost	3

Multiply two numbers (7*3) using Booth's algorithm

Step 2

A	Q $Q_3 Q_2 Q_1 Q_0$	Q_{-1}	Operation	Count
1100 	1001 	1	$Q_0 Q_{-1} = 11$ $ASR(A \ 001)$ Arithmetic Shift Right Loss	3
1110	0100	1		2
		1	$Q_0 Q_{-1} = 01$ $A = A + M$ $1110 + 0111$ $\begin{array}{r} 1110 \\ 0111 \\ \hline 10101 \end{array}$ Discarded $ASR(A \ 001)$ Loss	2
		0		1

Step 3

Multiply two numbers (7*3) using Booth's algorithm

A	Q $Q_3 Q_2 Q_1 Q_0$	Q_{-1}	Operation	Count
			$Q_0 Q_{-1} = 00$ $ASR(A, Q, Q_{-1})$	1
			$(0)1$	
	128643216 8421 0001 0101 $16+4+1=21$			

AKTU PYQ'S

Q1	Explain <u>booth's algorithm</u> in depth with the help of an <u>flowchart</u>. Give an example of multiplication of two number with the help of booth's algorithm.	AKTU 2016-17
Q2	Any <u>numerical</u> on booth's algorithm.	AKTU 2023-24 AKTU 2022-23 AKTU 2021-22 AKTU 2020-21 AKTU 2019-20 AKTU 2018-19

Computer organization and architecture

UNIT 2

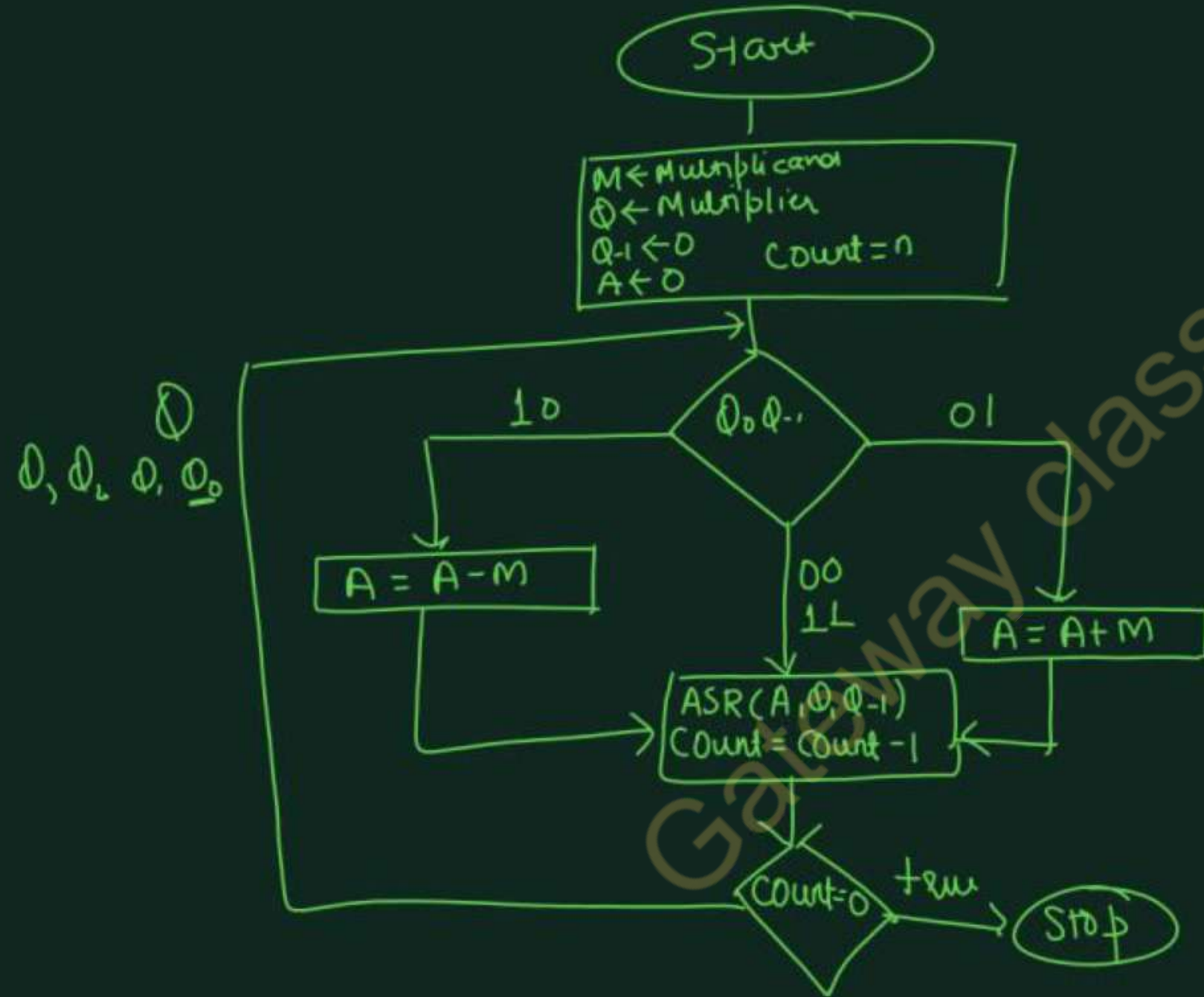
Arithmetic and logic unit

Today's Target

- Booth's algorithm Questions
- AKTU PYQs

By PRAGYA RAJVANSHI

B.Tech, M.Tech(C.S.E.)



$$-7 * 3$$

$$\begin{array}{l} M = (-7) \quad (1001) \\ Q = (3) \quad 0011 \end{array}$$

$$\text{Count} = 4$$

$$A = 0000$$

$$Q_1 = 0$$

$$-M = 0111 + 7$$

$$\begin{array}{r} +7 \\ 1's \\ 2's \end{array} \begin{array}{r} 8421 \\ 0111 \\ 1000 \\ +1 \\ \hline 1001 \end{array} (-7)$$

$$\begin{array}{r} 3 \\ 8421 \\ 0011 \end{array} \begin{array}{l} -M \\ -(-7) \end{array}$$

Q.1 Multiply two numbers (-7×3) using Booth's algorithm

A	Q $Q_3 Q_2 Q_1 Q_0$	Q_{-1}	Operation	Count
0000	0011	0	$Q_0 Q_{-1} = 10$ $A = A - M$ $A + (-M)$ $\begin{array}{r} 0000 \\ 0111 \\ \hline 0111 \end{array}$ $ASR(A, Q, Q_{-1})$	4
$\begin{array}{r} 0111 \\ \swarrow \downarrow \searrow \downarrow \\ 1001 \end{array}$	$\begin{array}{r} 0011 \\ \swarrow \downarrow \searrow \downarrow \\ 1100 \end{array}$	1	Lost	3

Q.1 Multiply two numbers (-7×3) using Booth's algorithm

A	Q	Q ₋₁	Operation	Count
		1	$Q_0 Q_{-1} = 11$ $ASR(A, Q_0, Q_{-1})$	3
0001	1100	1	$Q_0 Q_{-1} = 01$ $A = A + M$ $\begin{array}{r} 0001 \\ 1001 \\ \hline 1010 \end{array}$	2
		0	ASR	1

Q.1. Multiply two numbers (-7×3) using Booth's algorithm

A	Q	Q ₋₁	Operation	Count
		0	$Q_0 Q_{-1} = 00$ $ASR(A, Q, Q_{-1})$	1
		0	lost	0
	1110 1011 -21		Checking 1110 1011 6001 0100 1's Complement 1 2's Complement 0001 0101 128 64 32 16 8 4 2 1 16 + 4 + 1 = 21	

$$-7 * -3$$

$$M = -7 \text{ (1001)}$$

$$Q = -3 \text{ (1101)}$$

$$A = 0000$$

$$Q_{-1} = 0$$

$$\text{Count} = 4$$

$$(-M) = -(-7) = +7$$

↓

$$0111$$

$$\begin{array}{r} 8421 \\ 0111 \\ 1000 \\ +1 \\ \hline 1001 \end{array} \begin{array}{l} +7 \\ 15 \\ 2's \end{array}$$

$$\begin{array}{r} 8421 \\ 0011 \\ 1100 \\ +1 \\ \hline 1101 \end{array} \begin{array}{l} +3 \\ 15 \\ 2's \end{array}$$

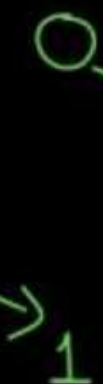
$$-8 + 4 + 1 = (-3)$$

$$A - M = A + (-M)$$

(-M)

$$-7 * -3 = 21$$

Q.2. Multiply two numbers (-7×-3) using Booth's algorithm

A	Q	Q ₋₁	Operation	Count
0000	1101	0	$Q_0 Q_{-1} = 10$ $A = A - M \quad (A + (-M))$	4
			$\begin{array}{r} 0000 \\ 0111 \\ \hline 0111 \end{array}$ $ASR(A, Q, Q_{-1})$ loss	3

Multiply two numbers (-7*-3) using Booth's algorithm

A	Q	Q ₋₁	Operation	Count
0011	1110	1	$Q_0Q_{-1} = 01$ $A = A + M$ $\begin{array}{r} 0011 \\ 1001 \\ \hline 1100 \end{array}$ $ASR(A, Q, Q_{-1})$	3
1100	1110	1	$Q_0Q_{-1} = 11$ $A = A - M$ $\begin{array}{r} 1100 \\ 1001 \\ \hline 0101 \end{array}$ $ASR(A, Q, Q_{-1})$	2



Lost

Multiply two numbers (-7*-3) using Booth's algorithm

A	Q	Q ₋₁	Operation	Count
1110	0111	0	$Q_0 Q_{-1} = 10$ $A = A - M$ $A + (-M)$ $\begin{array}{r} 1110 \\ \times 0111 \\ \hline 10101 \end{array}$ $ASR(A, Q, Q_{-1})$	2
			Lost	1

Multiply two numbers (-7×-3) using Booth's algorithm

A	Q	Q ₋₁	Operation	Count
		1	$Q_0 Q_{-1} = 11$ $ASR(A, Q_0 Q_{-1})$ 101	1
	128 64 32 16 8 4 2 1 00010101 21 $-7 \times -3 = 21$			

Computer organization and architecture

UNIT 2

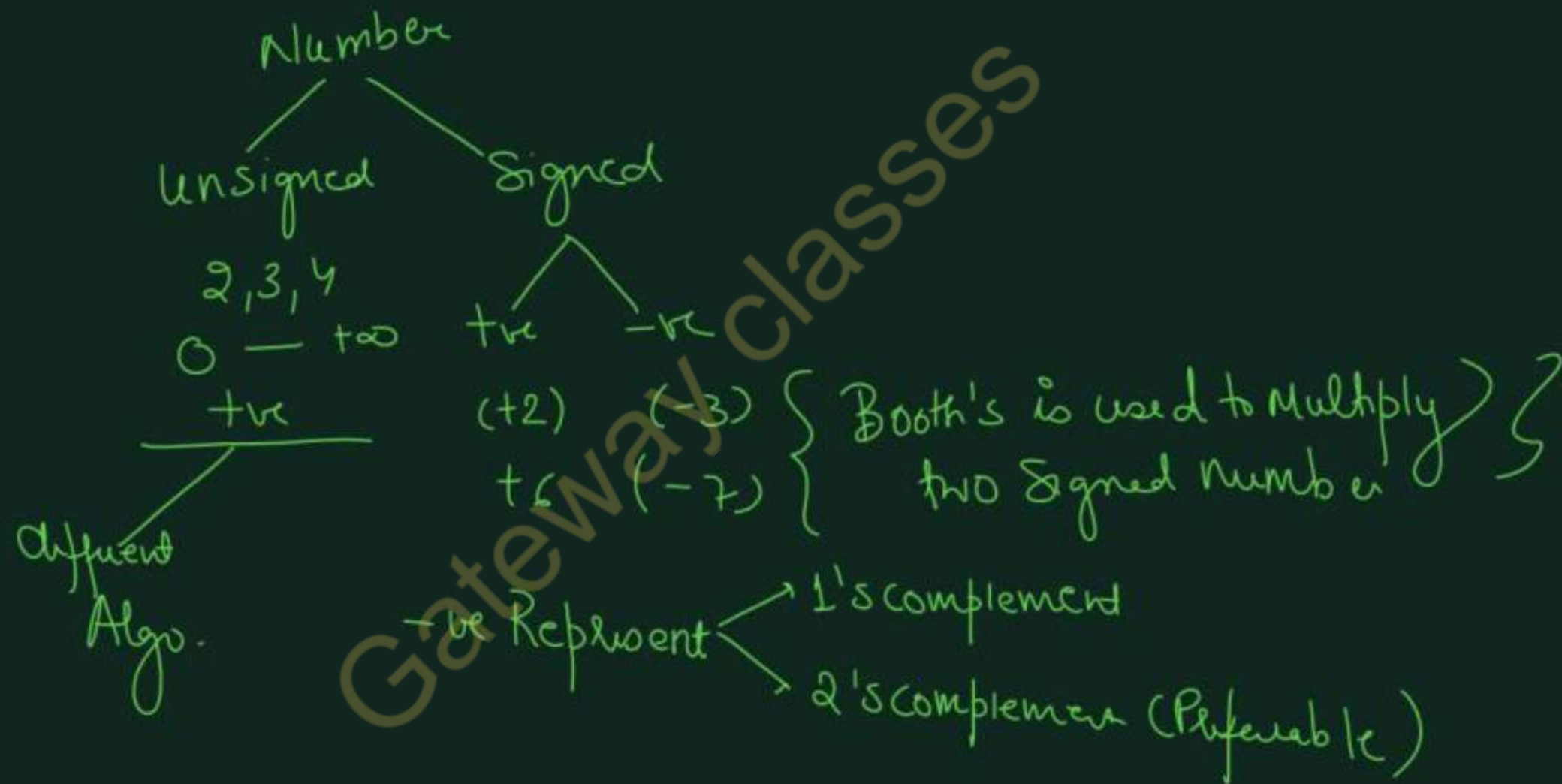
Arithmetic and logic unit

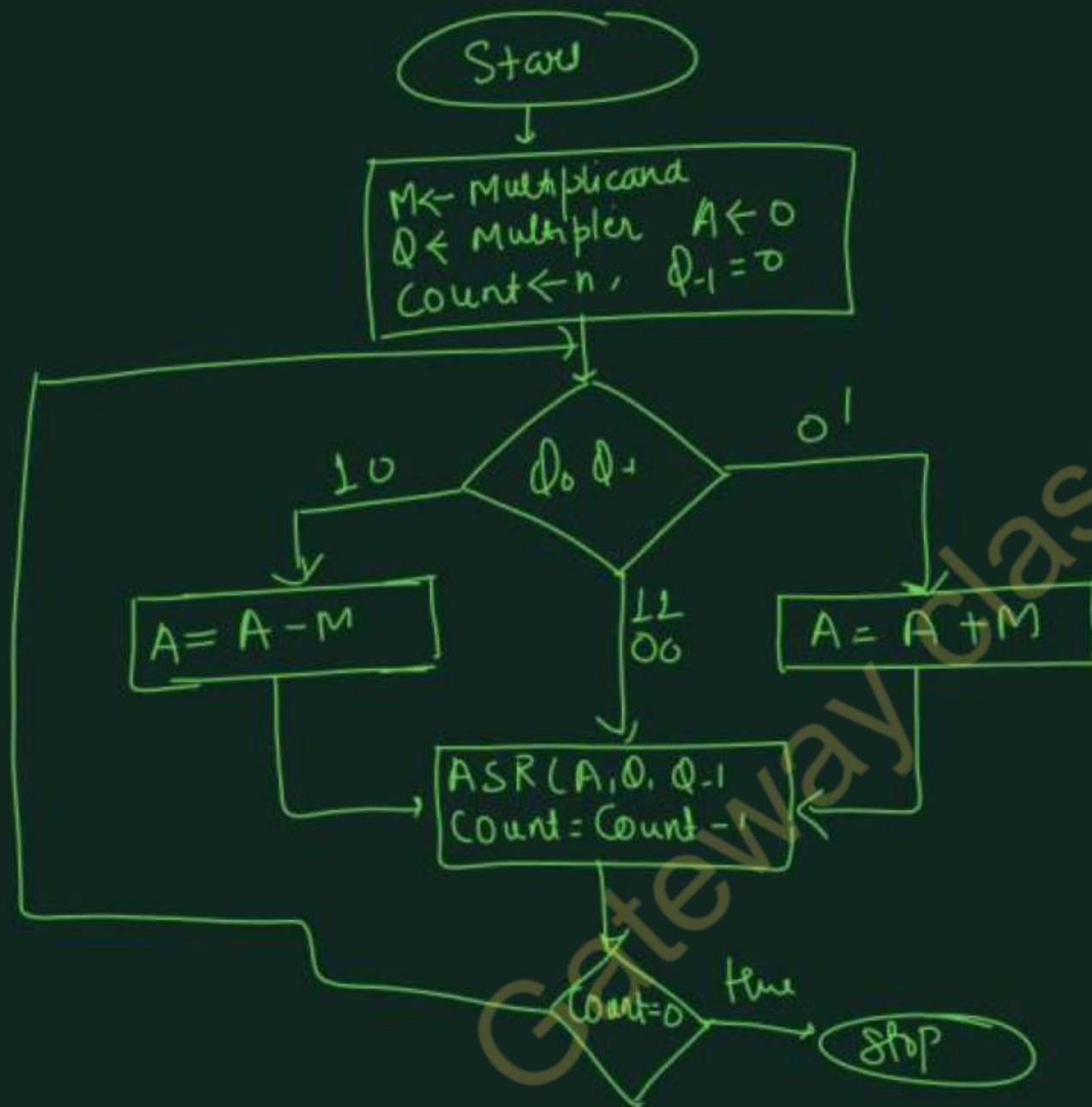
Today's Target

- Booth's algorithm Questions
- AKTU PYQ's

By PRAGYA RAJVANSHI

B.Tech, M.Tech(C.S.E.)





$M \leftarrow$ Multiplicand Register
 $Q \leftarrow$ Multiplier Register
 $Q_{-1} \leftarrow$ one bit special bit Register
 $A \leftarrow$ Accumulator Register
 Count (no of bits in $(M \& Q)$)

$$-15 * -16 = +240$$

M = -15	110001
Q = -16	110000
Count = 6	
A = 0000000	
Q ₋₁ ← 0	

$$-M = -(-15) = +15$$

$$-M = +15 = 001111$$

32	16	8	4	2	1	
0	1	0	0	0	0	+16
1	0	1	1	1	1	1's complement
				+	1	2's complement
						-16
1	1	0	0	0	0	

32	16	8	4	2	1	
0	0	1	1	1	1	+15
1	1	0	0	0	0	1's
				+	1	2's
1	1	0	0	0	1	

$$A = A - M$$

$$A + (-M)$$

Q.1 Multiply two numbers (-15×-16) using Booth's algorithm

Step 1

A	Q	Q ₋₁	Operation	Count
			$Q_0 Q_{-1} = 00$ $ASR(A, Q, Q_{-1})$ $LOS!$	6
			$Q_0 Q_{-1} = 00$ $ASR(A, Q, Q_{-1})$ $LOS!$	5
			$Q_0 Q_{-1} = 00$ $ASR(A, Q, Q_{-1})$ $LOS!$	4
			$Q_0 Q_{-1} = 00$ $ASR(A, Q, Q_{-1})$ $LOS!$	3

Step 2

Step 3

Q.1 Multiply two numbers (-15×-16) using Booth's algorithm

A	Q	Q ₋₁	Operation	Count
	Q_2, Q_1, Q_0 		$Q_0 Q_{-1} = 00 \text{ ASR}(A, Q, Q_{-1})$ 	3 2
			$Q_0 Q_{-1} = 10$ $A = A - M \rightarrow A = A + (-M)$ 	2
			$ASR(A, Q, Q_{-1})$ 	1

Q.1 Multiply two numbers (-15×-16) using Booth's algorithm

A	Q	Q ₋₁	Operation	Count
			$Q_0 Q_{-1} = 11$ $ASR(A, Q, Q_{-1})$ lost	1
	$128 \ 64 \ 32 \ 16 \ 8 \ 4 \ 2 \ 1$ 000011 11 0000 <u>Ans</u>		$\begin{array}{r} 128 \\ 64 \\ 32 \\ 16 \\ \hline + 240 \end{array}$	

Q.2 Multiply two numbers (+13*-15) using Booth's algorithm

A	Q	Q ₋₁	Operation	Count
00000	10001	0	$Q_0 Q_{-1} = 10$ $A = A - M$ $A + (-M)$	5
Step 1 <pre> 10011 11001 </pre>	<pre> 10001 11000 </pre>	0	<pre> 00000 10011 10011 </pre> ASR(A, Q, Q₋₁) lost	4

Q.2 Multiply two numbers (13*-15) using Booth's algorithm

A	Q	Q ₋₁	Operation	Count
Step 2 11001 00110 ↓ ↓ ↓ ↓ 00011	11000 11000 ↓ ↓ ↓ ↓ 01100	1	$Q_0Q_{-1} = 01$ $A = A + M$ $\begin{array}{r} 11001 \\ 01101 \\ \hline \otimes 00110 \end{array}$ $ASR(A, Q, Q_{-1})$ Lost	4
00011 ↓ ↓ ↓ ↓ 00001	01100 01100 ↓ ↓ ↓ ↓ 10110	0	$Q_0Q_{-1} = 00$ $ASR(A, Q, Q_{-1})$ Lost	3
		0		2

Q.2 Multiply two numbers (13*-15) using Booth's algorithm

A	Q	Q ₋₁	Operation	Count
Step 1		0	$Q_0 Q_{-1} = 00$ $ASR(A, Q, Q_{-1})$	2
Step 2		0	$Q_0 Q_{-1} = 10$ $A = A - M$ $\begin{array}{r} 00000 \\ 10011 \\ \hline 10011 \end{array}$ $ASR(A, Q, Q_{-1})$	1
		1	lost	0

Q.2 Multiply two numbers (13×-15) using Booth's algorithm

A	Q	Q ₋₁	Operation	Count
$ \begin{array}{r} \text{16 8 4 2 1} \\ \text{128 64 32} \\ \begin{array}{ c c c } \hline 1 & 0 & 0 \\ \hline \end{array} \\ \begin{array}{ c c c } \hline 0 & 0 & 1 & 1 & 0 \\ \hline \end{array} \\ \hline \begin{array}{ c c c } \hline 0 & 0 & 1 & 1 & 0 \\ \hline \end{array} \end{array} $	$ \begin{array}{r} \text{16 8 4 2 1} \\ \begin{array}{ c c c } \hline 1 & 1 & 1 & 0 & 1 \\ \hline \end{array} \\ \begin{array}{ c c c } \hline 0 & 0 & 0 & 1 & 0 & 1's \\ \hline \end{array} \\ \begin{array}{ c c c } \hline + & 1 & 2's \\ \hline \end{array} \\ \hline \begin{array}{ c c c } \hline 0 & 0 & 0 & 1 & 1 & +195 \\ \hline \end{array} \end{array} $	$ \begin{array}{r} 2 + 1 + 128 + 64 \\ \begin{array}{ c } \hline 128 \\ \hline \end{array} \\ \begin{array}{ c } \hline 64 \\ \hline \end{array} \\ \begin{array}{ c } \hline 3 \\ \hline \end{array} \\ \hline \begin{array}{ c } \hline 195 \\ \hline \end{array} \end{array} $	Ans (-195) $ \begin{array}{ c c c c c } \hline 1 & 1 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 1 \\ \hline \end{array} $	

Q.3 Multiply two numbers $(-13 * +8)$ using Booth's algorithm

A	Q	Q ₋₁	Operation	Count
Step 1 000000 000000	Q₀ 010000 000100	0 0	Q₀Q₋₁ = 00 ASR(A, Q₀, Q₋₁) Lost	5 4
Step 2 000000 000000	001000 000100	0 0	Q₀Q₋₁ = 00 ASR(A, Q₀, Q₋₁) Lost	4 3
Step 3 000000 000000	000100 000001	0 0	Q₀Q₋₁ = 00 ASR(A, Q₀, Q₋₁) Lost	3 2

Q.3 Multiply two numbers $(-13 * +8)$ using Booth's algorithm

A	Q	Q ₋₁	Operation	Count
000000	00001	0	$Q_0 Q_{-1} = 10$ $A = A - M = A + (-M)$ $\begin{array}{r} 00000 \\ 01101 \\ \hline 01101 \end{array}$	2
$\begin{array}{r} 01101 \\ \downarrow \downarrow \downarrow \downarrow \downarrow \\ 00110 \end{array}$	$\begin{array}{r} 00001 \\ \downarrow \downarrow \downarrow \downarrow \downarrow \\ 10000 \end{array}$	$\begin{array}{r} 0 \\ \downarrow \\ 1 \end{array}$	$ASR(A, Q, Q_{-1})$ Lost	1

Q.3 Multiply two numbers $(-13 * +8)$ using Booth's algorithm

A	Q	Q ₋₁	Operation	Count
00110	10000	1	$Q_0Q_{-1} = 01$ $A = A + M$ $\begin{array}{r} 00110 \\ 10011 \\ \hline 11001 \end{array}$	1
11001	10000	0	$ASR(A, Q, Q_{-1})$ $\rightarrow \text{Loss}$	0

Q.3 Multiply two numbers $(-13 \times +8)$ using Booth's algorithm

A	Q	Q ₋₁	Operation	Count
<u>Checking</u> $\begin{array}{r} 11100 \quad 11000 \\ 60011 \quad 00111 \\ \hline 00011 \quad 01000 \\ \hline 64 \quad 32 \quad 16 \quad 8 \quad 4 \quad 2 \quad 1 \end{array}$ $\begin{array}{r} -104 \\ 1's \\ 2's \\ +104 \end{array}$ If the 2's of a number is 104 then that is definitely = -104 $\begin{array}{r} 1 \\ 64 \\ 32 \\ 8 \\ \hline 104 \end{array}$ <u>Ans</u> $11100 \quad 11000$				

Logic Gate

AND Gate



A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

truth
table

OR Gate



A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

truth
table

NOT Gate



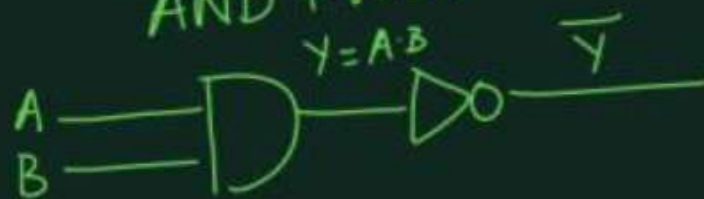
A	Y
0	1
1	0

truth table

NAND Gate

AND + NOT

$$Y = A \cdot B$$



A	B	$Y = A \cdot B$	$\overline{Y}$
0	0	0	1
0	1	0	1
1	0	0	1
1	1	1	0

Nand Gate



A	B	$\overline{Y}$
0	0	1
0	1	1
1	0	1
1	1	0

(NOR Gate
NOT+OR Gate



A	B	y	$\bar{y}$
0	0	0	1
0	1	0	1
1	0	0	1
1	1	0	1



A	B	y
0	0	0
0	1	1
1	0	1
1	1	1

AKTU Questions

Q.1

Show the multiplication process using booth's algorithm

1. $(-12) * (-18)$

2. $(+15) * (-13)$

3. $(+20) * (-19)$

4. $(-15) * (-16)$

5. $(+13) * (-15)$

6. $(+7) * (-3)$

7. $(-15) * (-16)$

8. $(-13) * (+8)$

9. $(-13) * (-21)$

10. $(-7) * (5)$

11. $(-13) * (8)$

12. $(-7) * (+3)$

(Signed number)

AKTU 2023-24

AKTU 2022-23

AKTU 2020-21

AKTU 2018-19

AKTU 2019-20

AKTU 2018-19

AKTU 2017-18

AKTU 2014-15

Q.2

Explain the booth's algorithm for multiplication of signed 2's complement number along with flowchart and a suitable example.

$(-3) * (-5)$

AKTU 2010-11

Computer organization and architecture

UNIT 2

Arithmetic and logic unit

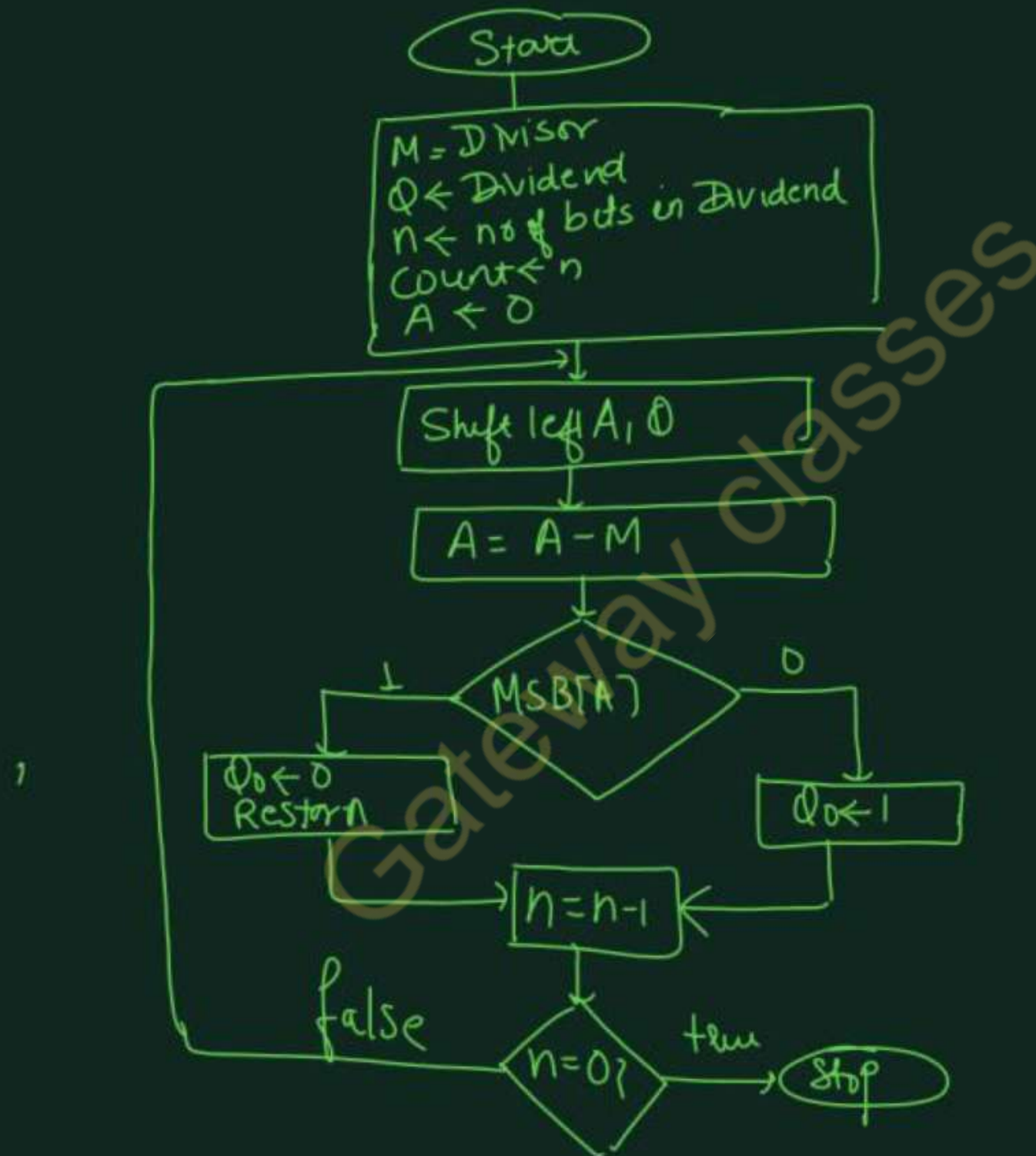
Today's Target

- Restoring division Algorithm.
- AKTU PYQs

By PRAGYA RAJVANSHI

B.Tech, M.Tech(C.S.E.)

Restoring Division Algorithm: for unsigned number



no of bits in Accumulator = $(n + L)$

Q.1 Restoring division Algorithm: for unsigned number

11(DIVIDEND)/3(Divisor)=3(Quotient) 2(Remainder)

$$(11)_{10} / (3)_{10}$$

$$Q = 11 \quad (1011)$$

$$M = 3 \quad (0011)$$

$$n = 4$$

$$\text{Count} = 4$$

$$A = 0000$$

$$-M(-3) = 1101$$

$$\begin{array}{r} 00011 \\ 11100 \text{ 1's} \\ \hline 11101 \text{ 2's} \end{array}$$

GATEWAY CLASSES

GATEWAY CLASSES

Restoring division Algorithm: for unsigned number 11/3

A	Q	Operation	Count
0000 00010 1111	0110 110 [1]	Shift left $A = A - M$ 00010 11101 ----- 11111 MSB[A] = 1 $Q_0 \leftarrow 0$ Restore A	3
00010	110 [?] 1100		2

Restoring division Algorithm: for unsigned number 11/3

A	Q	Operation	Count
$\begin{array}{r} 00010 \\ \swarrow \downarrow \downarrow \downarrow \swarrow \\ 00101 \end{array}$	$\begin{array}{r} 1100 \\ 100 \end{array} \boxed{?}$	Shift left $A = A - M$ $\begin{array}{r} 00101 \\ 11101 \\ \hline \otimes 00010 \end{array}$	2
$\begin{array}{r} 00010 \\ 00010 \end{array}$	$\begin{array}{r} 100 \end{array} \boxed{?}$	MSB[A] = 0 $Q_0 \leftarrow 1$	1
00010	1001		

Restoring division Algorithm: for unsigned number 11/3

A	Q Q_3, Q_2, Q_1, Q_0	Operation	Count
00010 $\swarrow \searrow \swarrow \searrow$ 00101 ← 10x 00010 00010 Remainder 2	1001 $\swarrow \searrow \swarrow \searrow$ 001 ? 001 ? 0011 Quotient 3	Shift left A, Q. $A = A - M$ 00101 11101 000010 MSB[A] = 0 $Q_0 \leftarrow 1$ 5421 0011	1 0

Restoring division Algorithm: for unsigned number 11/3

A	Q	Operation	Count
16 8 4 2 1 00010 Remainder - 2 84 21 0011 → 3 Quotient			

Q.2 Restoring division Algorithm: for unsigned number 0101/0011

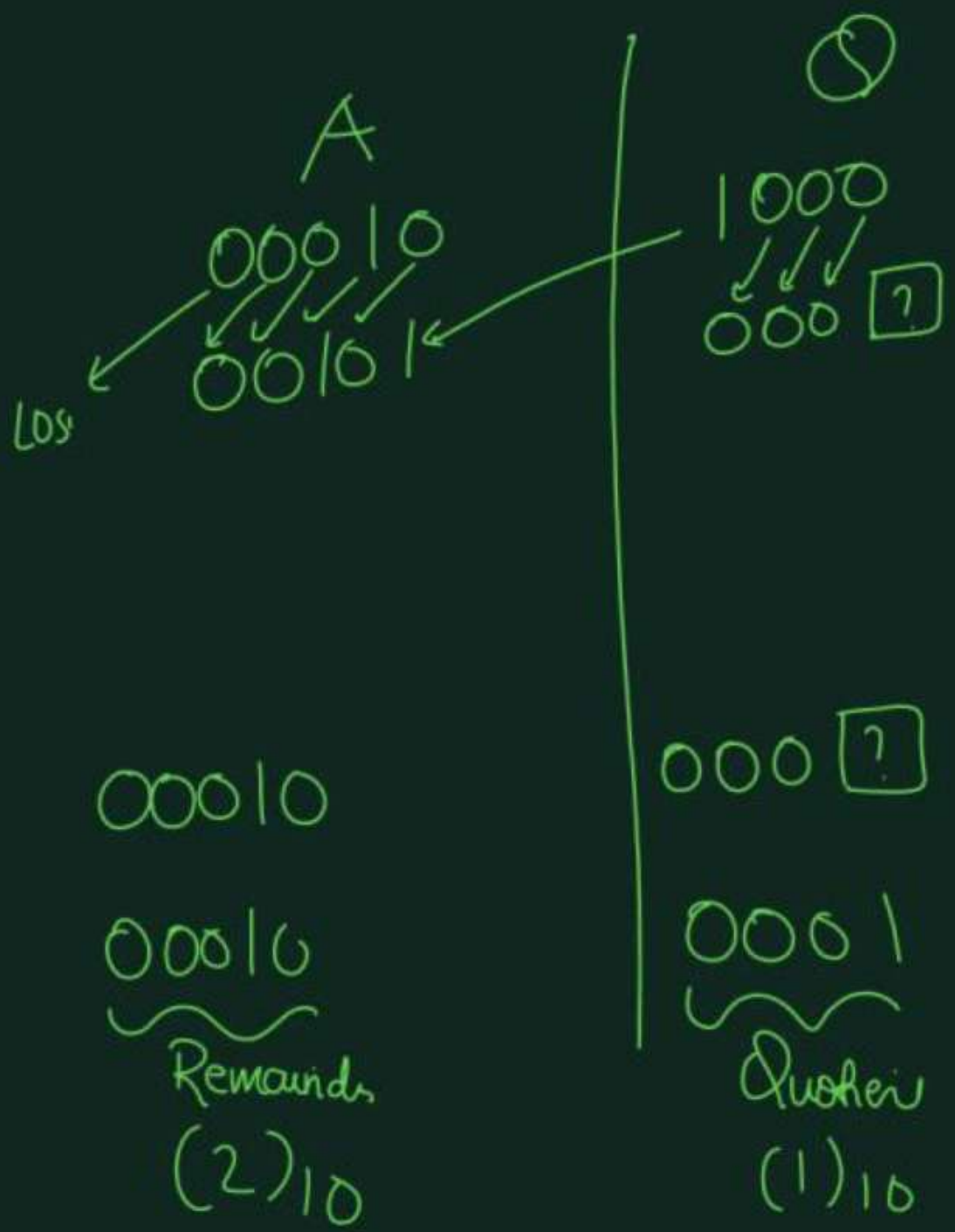
A	Q	Operation	Count
		Shift left A,Q $A = A - M \rightarrow A = A + (-M)$ $MSB[A] = 1$ $Q_0 \leftarrow 0$ Restore A	4 3

Restoring division Algorithm: for unsigned number 0101/0011

A	Q	Operation	Count
00000 00001 101 1110 00001	1010 010 1 010 1 0100	Shift left A, Q $A = A - M \rightarrow A = A + (-M)$ 00001 1101 1110 MSB[A] = 1 Restor A $Q_0 < 0$	3 2

Restoring division Algorithm: for unsigned number 0101/0011

A	Q	Operation	Count
0000 1 ↓ ↓ ↓ ↓ ↓ 1 0 0 0 1 0 1 0 0 0 1 0 0 0 0 0 0	0 0 0 1 0 0 [1] 1 0 0 [1] 1 0 0 0	Shift left A & Q $A = A - M$ 0 0 0 0 1 1 0 1 ——— 1 1 1 1 MSB[A] = 1 $Q_0 \leftarrow 0$ Restore A	2 1



Operation	Count
Shift left A, 0	
$A = A - M$	
$ \begin{array}{r} 00101 \\ 11101 \\ \hline \textcircled{X} 00010 \end{array} $	
MSB[A] = 0	
$Q_0 \leftarrow 1$	
	0

AKTU Questions

Q.1	Perform the division process of 00001111 by 0011 using division restore method.	AKTU 2023-24 AKTU 2021-22 AKTU 2018-19
Q.2	Multiply $(-8) * (-14)$ using <u>signed operand multiplication</u> ^(Booth's Algorithm) <u>method.</u>	AKTU 2020-21

Computer organization and architecture

UNIT 2

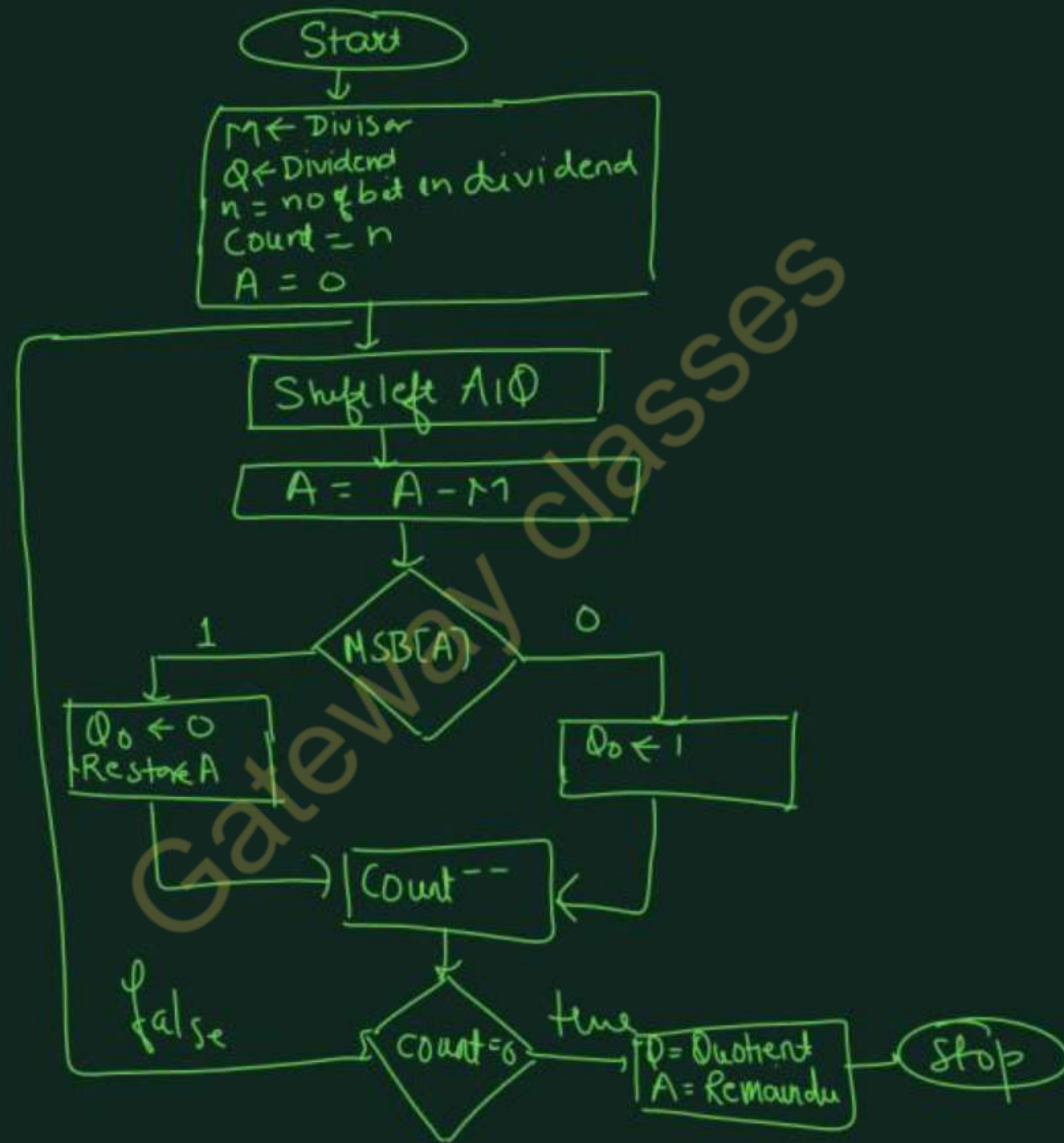
Arithmetic and logic unit

Today's Target

- Restoring and Non-Restoring division Algorithm.
- AKTU PYQs

By PRAGYA RAJVANSHI

B.Tech, M.Tech(C.S.E.)



no of bit in $Q_{cc} = n+1$

Q.1 Restoring division Algorithm: for unsigned number 00001111 by 0011

A	Q	Operation	Count
0000 0000 0 0000 0000 0 0 0000 0000 0	0000 0001 ? 0001 ? 0001 0	Shift left A, Q $A = A - M \Rightarrow A = A + (-M)$ 0000 0000 0 0 ----- 01 MSB[A] = 1 $Q_0 \leftarrow 0$ Restor A	8 7

Q.1 Restoring division Algorithm: for unsigned number 00001111 by 0011

A	Q	Operation	Count
		Shift left A, 0 $A = A - M \rightarrow A = A + (-M)$ $MSB[A] = 1$ $Q_0 \leftarrow 0$ Restore A	7
		$MSB[A] = 1$ $Q_0 \leftarrow 0$ Restore A	6

Q.1 Restoring division Algorithm: for unsigned number 00001111 by 0011

A	Q	Operation	Count
$\begin{array}{ccccccc} 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ & \swarrow & \downarrow & \downarrow & \downarrow & \downarrow & \swarrow \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{array}$ $\begin{array}{cccc} 1111 & 1110 & 1 & \\ \hline 0000 & 0000 & 0 & \end{array}$	$\begin{array}{ccc} 0011 & 1100 & \\ \hline 0111 & 1001 & \end{array}$ $\begin{array}{ccc} 011 & 100 & 1 \\ \hline 011 & 1000 & \end{array}$	Shift left A, Q. $A = A - M$ $\begin{array}{cccc} 0000 & 0000 & 0 & \\ 1111 & 1110 & 1 & \\ \hline 1111 & 1110 & 1 & \end{array}$ MSB[A] = 1 $Q_0 \leftarrow 0$ Restor A	6 5

Q.1 Restoring division Algorithm: for unsigned number 00001111 by 0011

A	Q	Operation	Count
		Shift left A, Q $A = A - M \Rightarrow A = A + (-M)$ MSB[A] = 1 $Q_6 \leftarrow 0$ Restor A	5 4

Q.1 Restoring division Algorithm: for unsigned number 00001111 by 0011

A	Q	Operation	Count
0000 0000 0 0000 0000 1 0 0000 0000 1	0000 0000 0000 0000	Shift left A, Q $A = A - M \Rightarrow A = A + (-M)$ 0000 0000 1 0 ----- 1 0 MSB[A] = 1 $Q_0 \leftarrow 0$ Restor A	4 3

Q.1 Restoring division Algorithm: for unsigned number 00001111 by 0011

A	Q	Operation	Count
0000 0000 1 0000 0000 1	11 0 0000 11 00 000 1	Shift left A, Q $A = A + (-M)$ 0000 000 1 1 11 1 1 1 0 1 ----- ⊗ 00 00 000 0 0 MSB[A] = 0 $Q_0 \leftarrow 1$	3
0000 0000 0 0000 0000 0	11 00 000 1 11 00 000 1		2

Q.1 Restoring division Algorithm: for unsigned number 00001111 by 0011

A	Q	Operation	Count
		Shift left A, Q $A \leftarrow A - M \rightarrow A = A + (-M)$ MSB[A] = 1 $Q_0 \leftarrow 0$ Restore A	2
			1

Q.1 Restoring division Algorithm: for unsigned number 00001111 by 0011

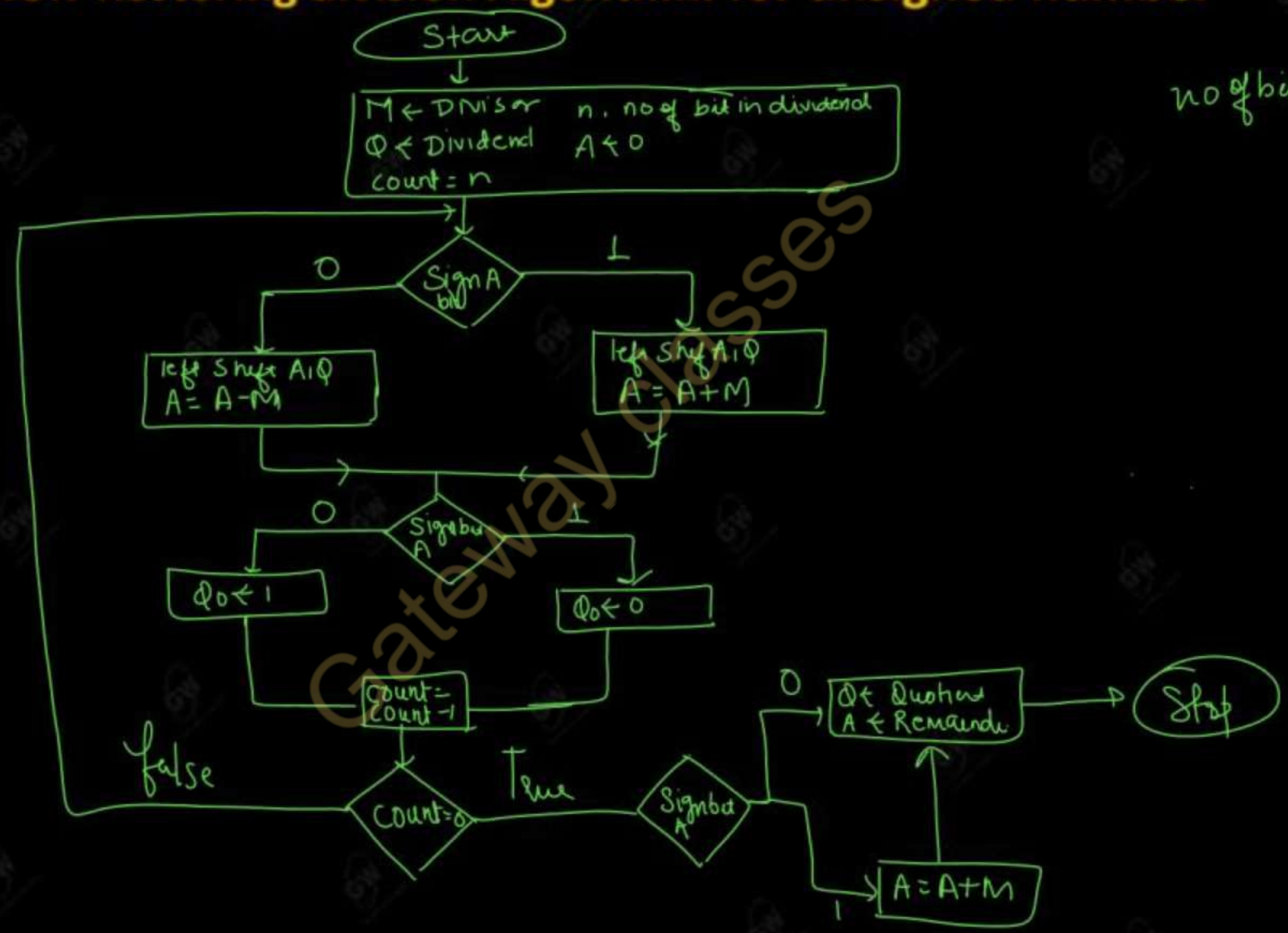
15
8 7 6 5 4 3 2 1

A	Q	Operation	Count
0000 0000 0000 0001 0000 0000 0 0000 0000 0 Remainder (0)₁₀	1000 0010 0000 0101 0000 0101 (5)₁₀	Shift left A, Q $A = A - M \rightarrow A = A + (-M)$ 0000 0001 1111 1110 0000 0000 0 MSB[A] = 0 $Q_0 \leftarrow 1$	1 0

Non-Restoring division Algorithm: for unsigned number

no of bit in Acc = $n+1$

0010
↑
MSB /
Signbit
LSB



Q.2 Non-Restoring division Algorithm: for unsigned number 11/3

A	Q	Operation	Count
00000 00001	1011 011 □	Sign bit [A] = 0 Shift left A, Q $A = A - M \rightarrow A = A + (-M)$ <pre> 00001 11101 ----- 11110 </pre>	4
11110 11110	011 □ 0110	Sign bit [A] = 1 $Q_0 \leftarrow 0$	3

Q.2 Non-Restoring division Algorithm: for unsigned number 11/3

A	Q	Operation	Count
$\begin{array}{r} 11110 \\ 11100 \\ \hline 111000 \end{array}$	$\begin{array}{r} 0110 \\ 110 \end{array}$	Sign bit $[A] = 1$ Shift left A, Q. $A = A + M$ $\begin{array}{r} 11100 \\ 00011 \\ \hline 11111 \end{array}$	3
$\begin{array}{r} 1111 \\ 1111 \end{array}$	$\begin{array}{r} 110 \\ 1100 \end{array}$	Sign Bit $[A] = 1$ $Q_0 \leftarrow 0$	2

1. ?

Q.2 Non-Restoring division Algorithm: for unsigned number 11/3

A	Q	Operation	Count
00010 00101	1001 001□	Signbit[A] = 0 Shift left A, 1 $A = A - M \rightarrow A = A + (-M)$ $\begin{array}{r} 00101 \\ 11101 \\ \hline \otimes 00010 \end{array}$	1
00010 00010 00010 Remainder	001□ 0011 0011 Quotient	Signbit[A] = 0 $Q_0 \leftarrow 1$ Sign[A] = 0	0

Q.4 Non-Restoring division Algorithm: for unsigned number 1010 by 0101

A	Q	Operation	Count
00000	10 0	Signbit[A] = 0	4
00001	0 0□	Shift left A, Q	
		$A = A - M \rightarrow A = A + (-M)$	
11100	0 0□	$\begin{array}{r} 00001 \\ 11011 \\ \hline 11100 \end{array}$	
11100	0 00	Signbit[A] = 1	3
		$Q_0 \leftarrow 0$	

Q.4 Non-Restoring division Algorithm: for unsigned number 1010 by 0101

A	Q	Operation	Count
11100 11111 11000 11101 11101	0100 100? 100? 1000	Signbit[A] = 1 Shift left A 10 A = A + M 11000 00101 11101 Signbit[A] = 1 Q₀ ← 0	3 2

Q.4 Non-Restoring division Algorithm: for unsigned number 1010 by 0101

A	Q	Operation	Count
1110 1101 00000 00000	1000 - 000 000 000 000 000	Sign bit [A] = 1 Shift left A, Q A = A + M 1101 0010 00000 ⊗ 00000 Sign [A] = 0 Q₀ ← 1	2 1

Q.4 Non-Restoring division Algorithm: for unsigned number 1010 by 0101

A	Q	Operation	Count
00000 00000 11011 11011	0001 001? 001? 0010	Sign bit [A] = 0 Shift left A $A = A - M \rightarrow A = A + (-M)$ 00000 11011 11011 Sign bit [A] = 1 $Q_0 \leftarrow 0$	1 0

A	①	Operation	Count
11011	0010	Signbit[A] = 1 $A = A + M$ $\begin{array}{r} 11011 \\ 00101 \\ \hline \textcircled{\times} 00000 \end{array}$	0
00000 Remainder	0010 Quotient		

AKTU Questions

Q.1

Perform the division process of $(11)_{10}$ by $(3)_{10}$ using division non-restore method.

AKTU 2019-19

AKTU 2020-21

$$\begin{array}{r|l} 1000 & 0011 \\ \hline \end{array}$$

 Dividend (Represent 7 bit)

$$\begin{array}{r|l} 000 & 000 \\ \hline \end{array} \quad \begin{array}{r|l} 0000 & 0011 \\ \hline \end{array}$$

 $A = 00000000$

 M

 Count = 7

$$\begin{array}{r|l} 12 & 3 \\ \hline \end{array} = 4 \quad 0$$

 Quotient

 Remainder

$$\begin{array}{r|l} 1000 & 0011 \\ \hline \end{array}$$

 $8 | 3 = 2 \quad 2$

Computer organization and architecture

UNIT 2

Arithmetic and logic unit

Today's Target

- IEEE Standard 754 Floating Point Numbers
- Numericals on floating point number
- AKTU PYQs

By PRAGYA RAJVANSHI

B.Tech, M.Tech(C.S.E.)

IEEE Standard 754 Floating Point Numbers

- The IEEE Standard for Floating-Point Arithmetic (IEEE 754) is a technical standard for floating-point computation which was established in 1985 by the Institute of Electrical and Electronics Engineers (IEEE)

1. The Sign of Mantissa –

- This is as simple as the name. 0 represents a positive number while 1 represents a negative number.

2. The Biased exponent –

- The exponent field needs to represent both positive and negative exponents. A bias is added to the actual exponent in order to get the stored exponent.

3. Normalizing the mantissa

It means adjusting a floating-point number so that the leading digit of the mantissa is non-zero (usually 1 in binary). This ensures the number is in standard form, maximizing precision

The binary number **0.01101** is normalized to **1.101×2^{-2}** by shifting the decimal point and adjusting the exponent.

Single precision IEEE 754 floating point standard

← 32 →

Sign(1)	Exponent (8)	Mantissa(23)
---------	--------------	--------------

Total bit=32 bits

Double precision IEEE 754 floating point standard

Sign(1)	Exponent(11)	Mantissa(52)
---------	--------------	--------------

Total bits=64

Represent the floating pointing number using IEEE standard both single and double precision

1.(314.175)₁₀

2	314	
2	157	0
2	78	1
2	39	0
2	19	1
2	9	1
2	4	1
2	2	0
2	1	0
		1

① Convert it into binary

(100111010)₂

(314.175)₁₀

(100111010.0010110)₂

175 * 2 = 0.35 → 0
 35 * 2 = 0.7 → 0
 7 * 2 = 1.4 → 1
 4 * 2 = 0.8 → 0
 8 * 2 = 1.6 → 1
 16 * 2 = 1.2 → 1
 12 * 2 = 0.4 → 0

(0010110)₂

0.4 * 2 = 0.8 → 0
 0.8 * 2 = 1.6 → 1
 1.6 * 2 = 1.2 → 1
 0.2 * 2 = 0.4 →

$$(100111010.0010110)_2$$

② Normalisation

$$1.001110100010110 \times 2^8$$

Single bit precision

$$1.N \times 2^{E-127}$$

$$E-127=8$$

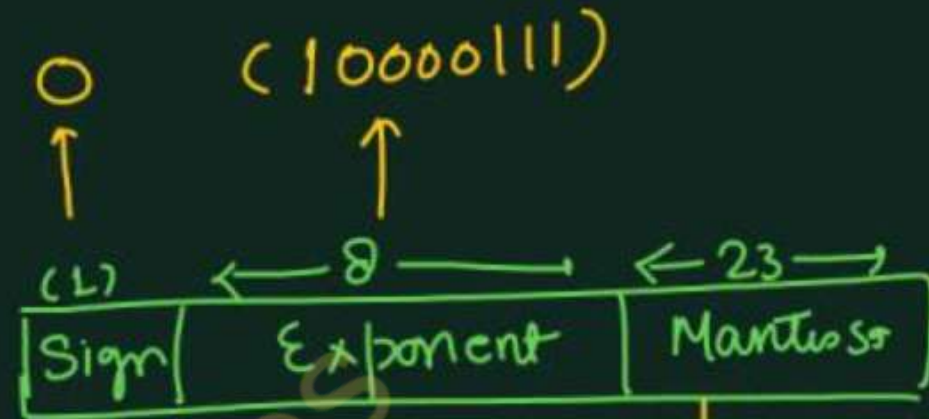
$$E=127+8$$

$$135$$

$$(135)_{10}$$

$$(10000111)_2$$

2	135	
2	67	1
2	33	1
2	16	1
2	8	0
2	4	0
2	2	0
2	1	0
		1



double precision

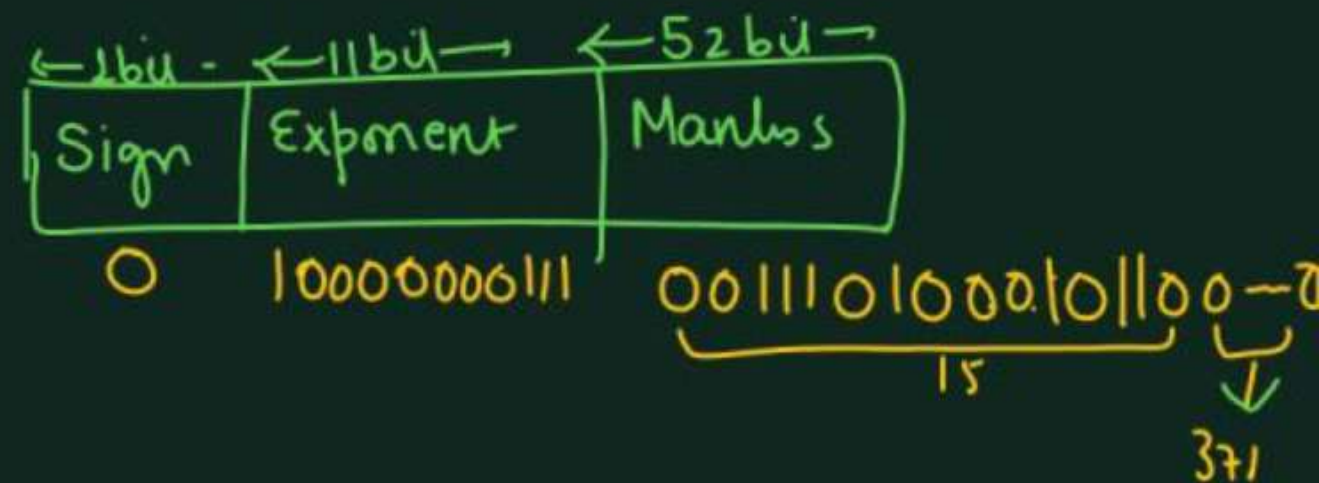
$$1.N * 2^{E-1023}$$

$$E-1023 = 8$$

$$E = 1023 + 8$$

2	1031	
2	515	1
2	257	1
2	128	1
2	64	0
2	32	0
2	16	0
2	8	0
2	4	0
2	2	0
2	1	0
		1

$$(10000000111)_2$$



Represent the floating pointing number using IEEE standard both single and double precision

2. $(-786.25)_{10}$

786

2	786	
2	393	0
2	196	1
2	98	0
2	49	0
2	24	1
2	12	0
2	6	0
2	3	0
2	1	1
		1

$(1100010010)_2$

$$.25 * 2 = 0.5 \rightarrow 0$$

$$.5 * 2 = 1.0 \rightarrow 1$$

$$0 * 2 = 0$$

$(786.25)_{10}$

$(1100010010.01)_2$

① Normalise

$$1.10001001001 * 2^9$$

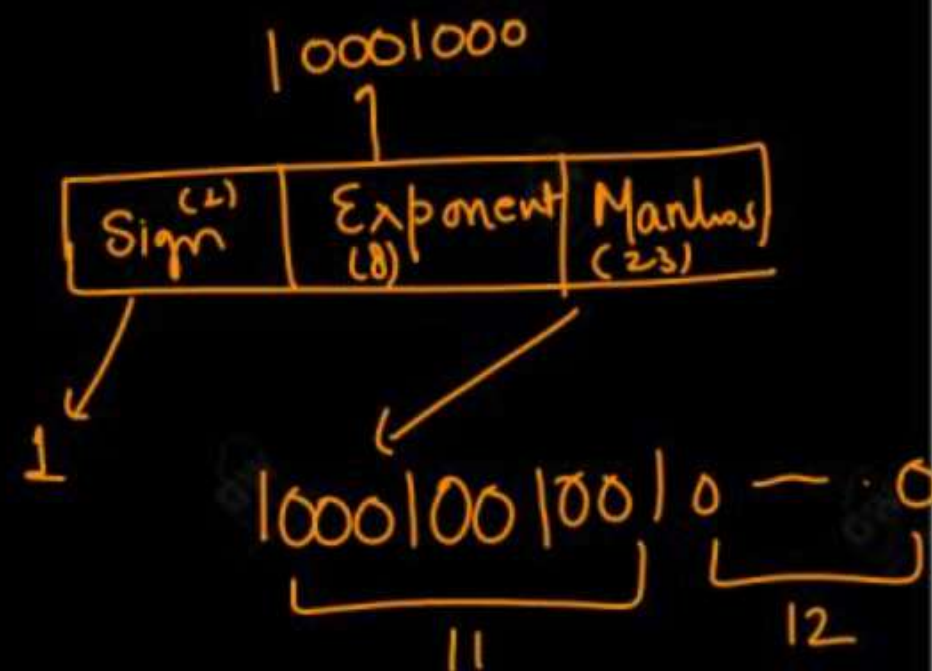
Single bit precision

$$1.N * 2^{E-127}$$

$$E-127 = 9 \rightarrow 136$$

2	136	
2	68	0
2	34	0
2	17	0
2	8	1
2	4	0
2	2	0
2	1	0
		1

$(10001000)_2$



$$1.N \times 2^{E-1023}$$

$$E-1023 = 9$$

$$E = 1023 + 9 = 1032$$

2	1032	
2	516	0
2	258	0
2	129	0
2	64	1
2	32	0
2	16	0
2	8	0
2	4	0
2	2	0
2	1	0
		1

$$(10000001000)_2$$

$$1000|00|00|0 \dots 0$$

Sign (1)	Exponent (11)	Mantissa (52)
-------------	------------------	------------------

$$(1) \quad 1000000|000$$

Computer organization and architecture

UNIT 2

Arithmetic and logic unit

Today's Target

- Multiplication: unsigned number
- AKTU PYQs

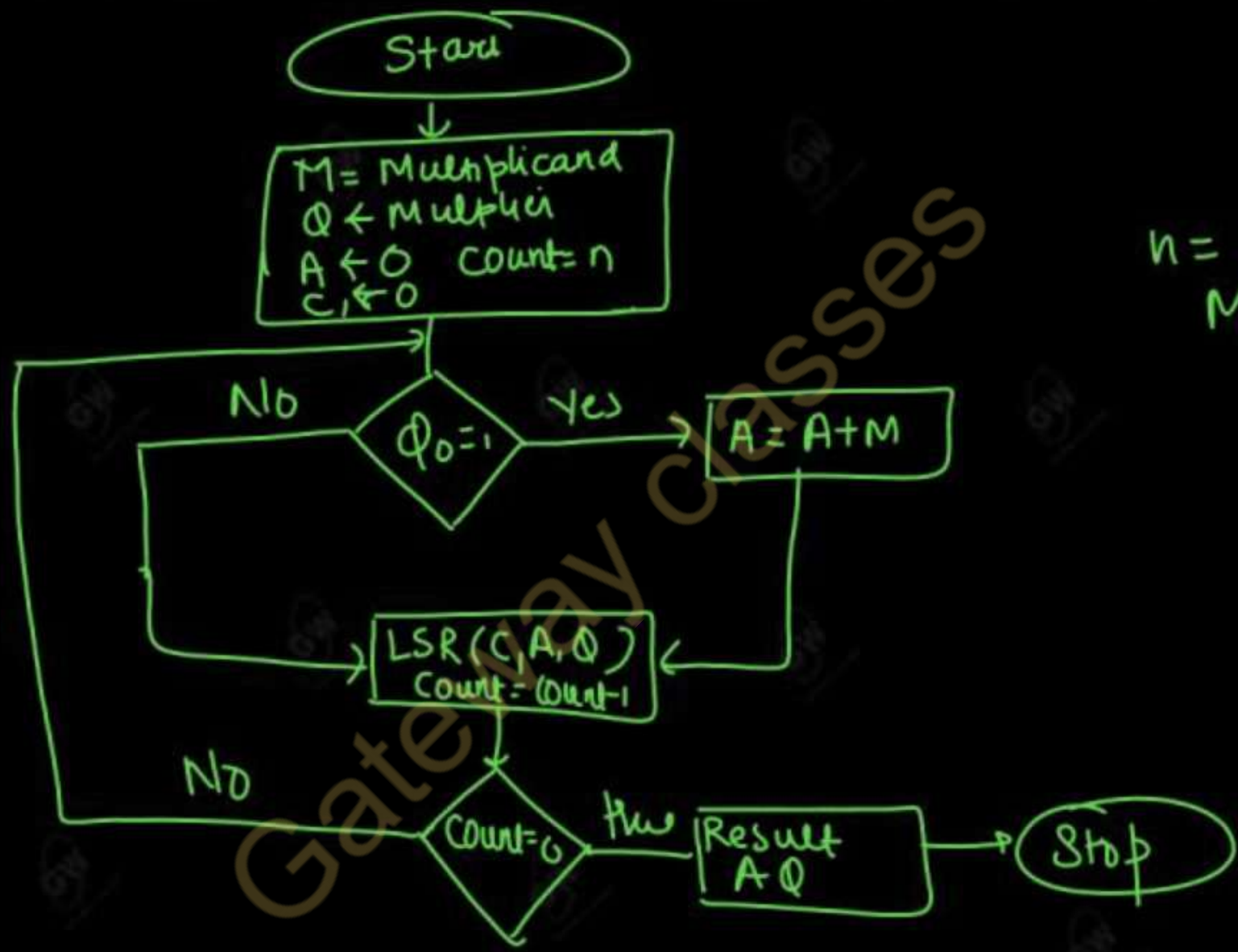
By PRAGYA RAJVANSHI

B.Tech, M.Tech(C.S.E.)

Multiplication for unsigned number

(Sequential Multiplication)

n = number of bit in Multiplier or Multiplicand



$$\underline{11 * 13 = 143}$$

$$1011 * 1101$$

$$(11) M = 1011$$

$$(13) D = 1101$$

$$A = 0000$$

$$C = 0$$

$$\text{Count} = 4$$

8	4	2	1	→	11
1	0	1	1	→	11
1	1	0	1	→	13
1					

Multiply two unsigned number 1011*1101

C	A	Q ₀	OPERATION	Count
0	0000	1101	$Q_0 = 1$ $A = A + M$ $\begin{array}{r} 0000 \\ 1011 \\ \hline 1011 \end{array}$	4
0	1011	1101	$LSR(C, A, 1)$ Lost	3



Multiply two unsigned number 1011*1101

C	A	Q	OPERATION	Count
0	0101	1110	$Q_0 = 0$ $LSR(C, A, Q)$	3
0	0010	1111	Loss	2
0	0010	1111	$Q_0 = 1$ $A = A + M$	2
			$\begin{array}{r} 0010 \\ 1011 \\ \hline 1101 \end{array}$	
0	1101	1111	$LSR(C, A, Q)$	
0	0110	1111	Loss	1

Multiply two unsigned number 1011*1101

C	A	Q	OPERATION	Count
0	0110	1111	$Q_0 = 1$ $A = A + M$ $\begin{array}{r} 0110 \\ 1011 \\ \hline 10001 \end{array}$ LSR (C, A, Q) 1011	1
0			$Q_0 = 1$ $A = A + M$ $\begin{array}{r} 0110 \\ 1011 \\ \hline 10001 \end{array}$ LSR (C, A, Q) 1011	0

Multiply two unsigned number 1011*1101

C	A	Q	OPERATION	Count
	128 64 32 16 8 4 2 1 1000 1111 $128 + 8 + 4 + 2 + 1 = 143$			

Multiply two unsigned number 6*14

C	A	Q	OPERATION	Count

Gateway classes

$$6 * 14 = 84$$

$$M = 0110$$

$$D = 1110$$

$$A = 0000$$

$$C \leftarrow D$$

$$\text{Count} = 4$$

$$8421$$

$$1110$$

$$8421$$

$$0110$$

Multiply two unsigned number 6*14

C	A	Q	OPERATION	Count
0	0000	1110	$Q_0 = 0$ LSR(CC, A, Q)	4
0	0000	0111	lost	3
0	0000	0111	$Q_0 = 1 \rightarrow A = A + M$	3
0	0110	0111	0000 0110 0110 LSR(CC, A, Q) lost	2

Multiply two unsigned number 6*14

C	A	Q	OPERATION	Count
0	0011	0011	$Q_0 = 1$ $A = A + M$ $\begin{array}{r} 0011 \\ 0110 \\ \hline 1001 \end{array}$ $LSR(C, A, Q)$	2
0			$Lost$	1

C	A	Q	OPERATION	Count
0	0100	1001	$\Phi_0 = 1$ $A = A + M$ $\begin{array}{r} 0100 \\ 0110 \\ \hline 1010 \\ \text{LSR}(C, A, \Phi) \end{array}$	1
0 → 0	$\begin{array}{c} 1010 \\ \swarrow \downarrow \searrow \\ 0101 \end{array}$ 64 32 16 8 4 2 1 01010100 - 84 $64 + 16 + 4 = 84$	$\begin{array}{c} 1001 \\ \swarrow \downarrow \searrow \\ 0100 \end{array}$	Lost	0

UNIT 2

Arithmetic and logic unit

Today's Target

- Half adder and Full adder
- AKTU PYQs

By PRAGYA RAJVANSHI

B.Tech, M.Tech(C.S.E.)

HALF ADDER

- It is a **digital logic circuit** that perform **binary addition of two single bit binary number**
- Sum is **LSB**(LEAST SIGNIFICANT BIT)
- CARRY IS **(MOST SIGNIFICANT BIT)**
- For K MAP **n** variable the **2ⁿ cells**

0 - B'
1 - B
0 - A'
1 - A

	^{2¹} A	^{2⁰} B	SUM	CARRY
0	0	0	0	0
1	0	1	1	0
2	1	0	1	0
3	1	1	0	1

	A	$\bar{A}$
B	0	1
$\bar{B}$	0	1
B	1	0

$A\bar{B} + \bar{A}B$
SUM = $A \oplus B$

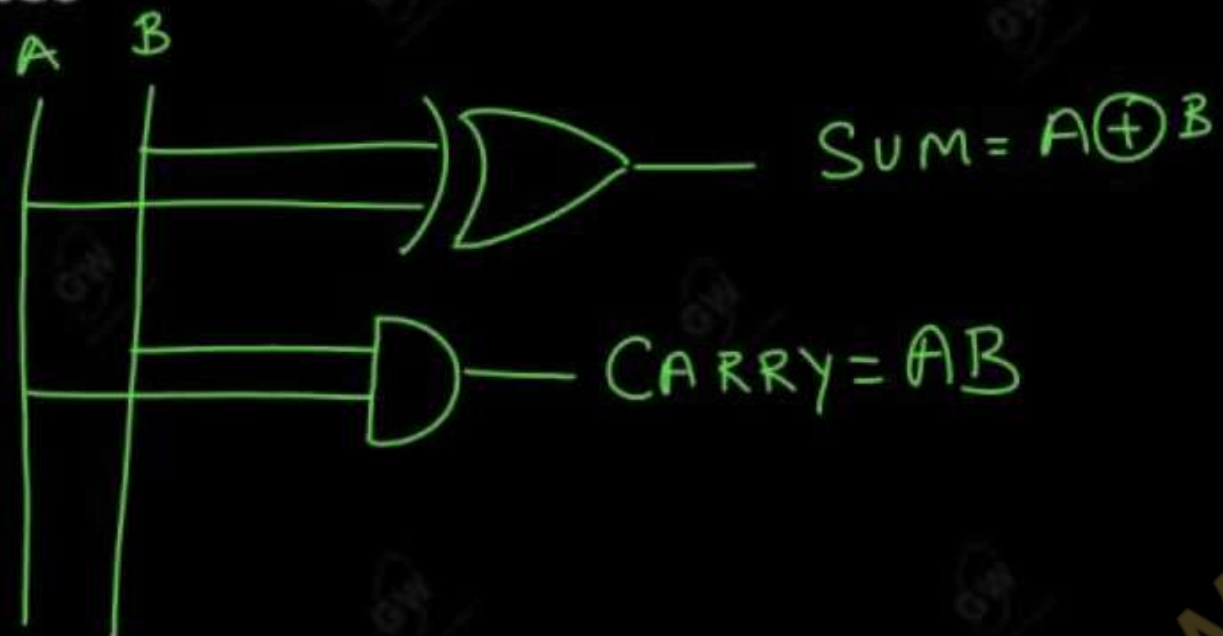
Carry = AB

	$\bar{A}$	A
$\bar{B}$	0	1
B	1	0

IMPLEMENTATION

half adder is implemented by XOR and AND

gates



$$\begin{aligned} \text{Sum} &= A \oplus B \\ \text{Carry} &= AB \end{aligned}$$

Block Diagram



FULL ADDER

- It solve the problem of **half adder**(not adding carry bit).
- Full adder is used to add **three 1 bit number**.
- A,B,Cin(carry)

Input			Output	
A	B	C _{in}	SUM	CARRY
0	0	0	0	0
1	0	1	1	0
2	1	0	1	0
3	1	1	0	1
4	0	0	1	0
5	0	1	0	1
6	1	0	0	1
7	1	1	1	1

FULL ADDER (for sum)

Cin	AB	$\bar{A}\bar{B}$	$\bar{A}B$	AB	$A\bar{B}$
		00	01	11	10
Cin 0		0	1	2	4
Cin 1	1		3	7	5

$$\begin{aligned}
 &\bar{A}\bar{B}C_{in} + \bar{A}B\bar{C}_{in} + AB C_{in} + A\bar{B}\bar{C}_{in} \\
 &\bar{A}(\bar{B}C_{in} + B\bar{C}_{in}) + A(\bar{B}C_{in} + B\bar{C}_{in}) \\
 &\bar{A}(B \oplus C) + A(\bar{B} \oplus \bar{C}) \\
 &\bar{A}X + A\bar{X} \\
 &A \oplus X \rightarrow \boxed{A \oplus B \oplus C}
 \end{aligned}$$

$$\begin{aligned}
 A \oplus B &= A\bar{B} + \bar{A}B \\
 \underline{(A \oplus B)} &= AB + \bar{A}\bar{B}
 \end{aligned}$$

FULL ADDER (for carry)

Cin	AB 00	$\bar{A}\bar{B}$ 01	$\bar{A}B$ 11	$A\bar{B}$ 10
Cin 0	0	2	4	4
Cin 1	1	3	4	5

$$\begin{aligned}
 &AB\bar{C}_{in} + ABC_{in} \\
 &AB(\bar{C}_{in} + C_{in}) \\
 &AB \cdot 1 = AB
 \end{aligned}$$

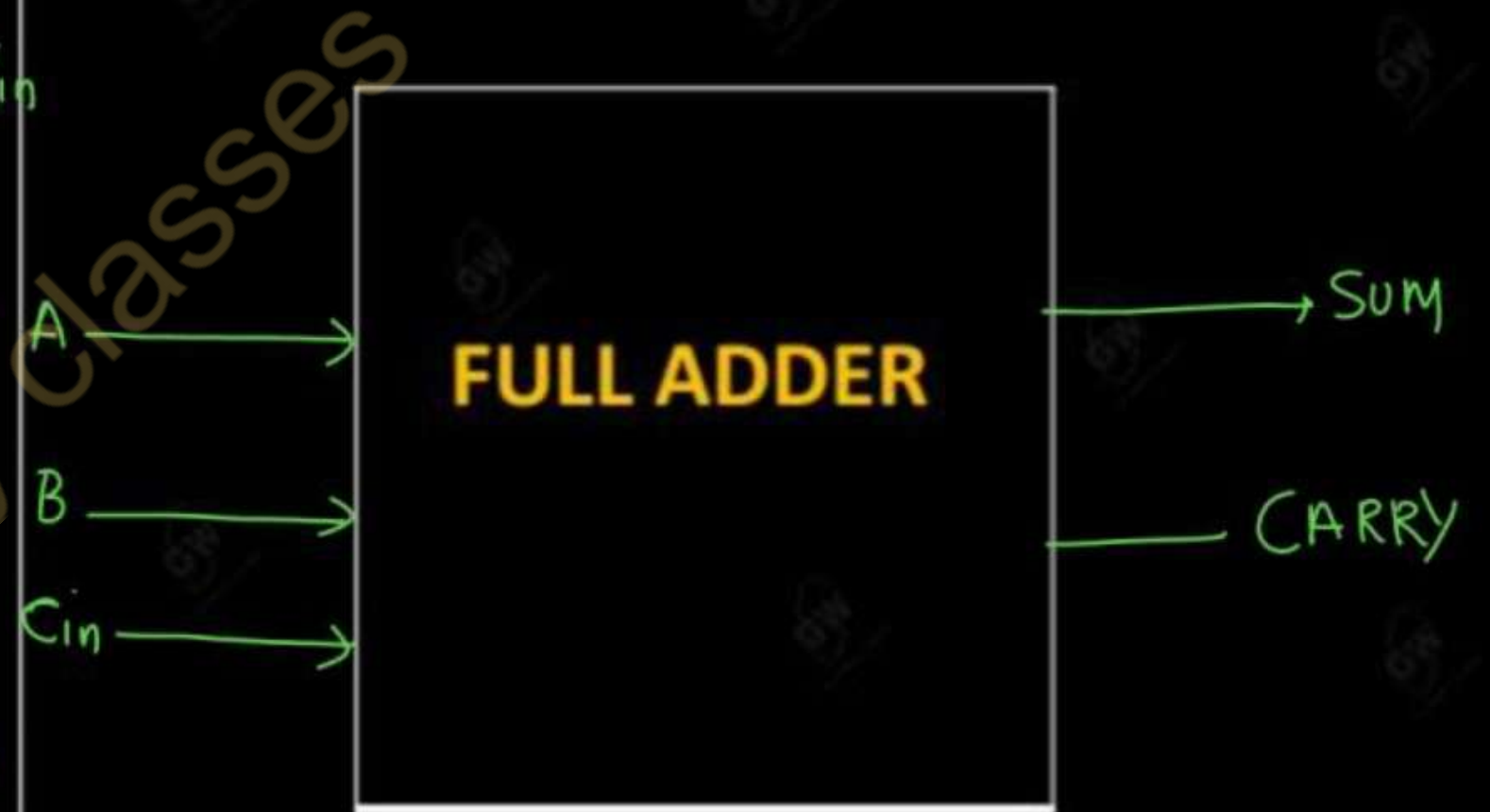
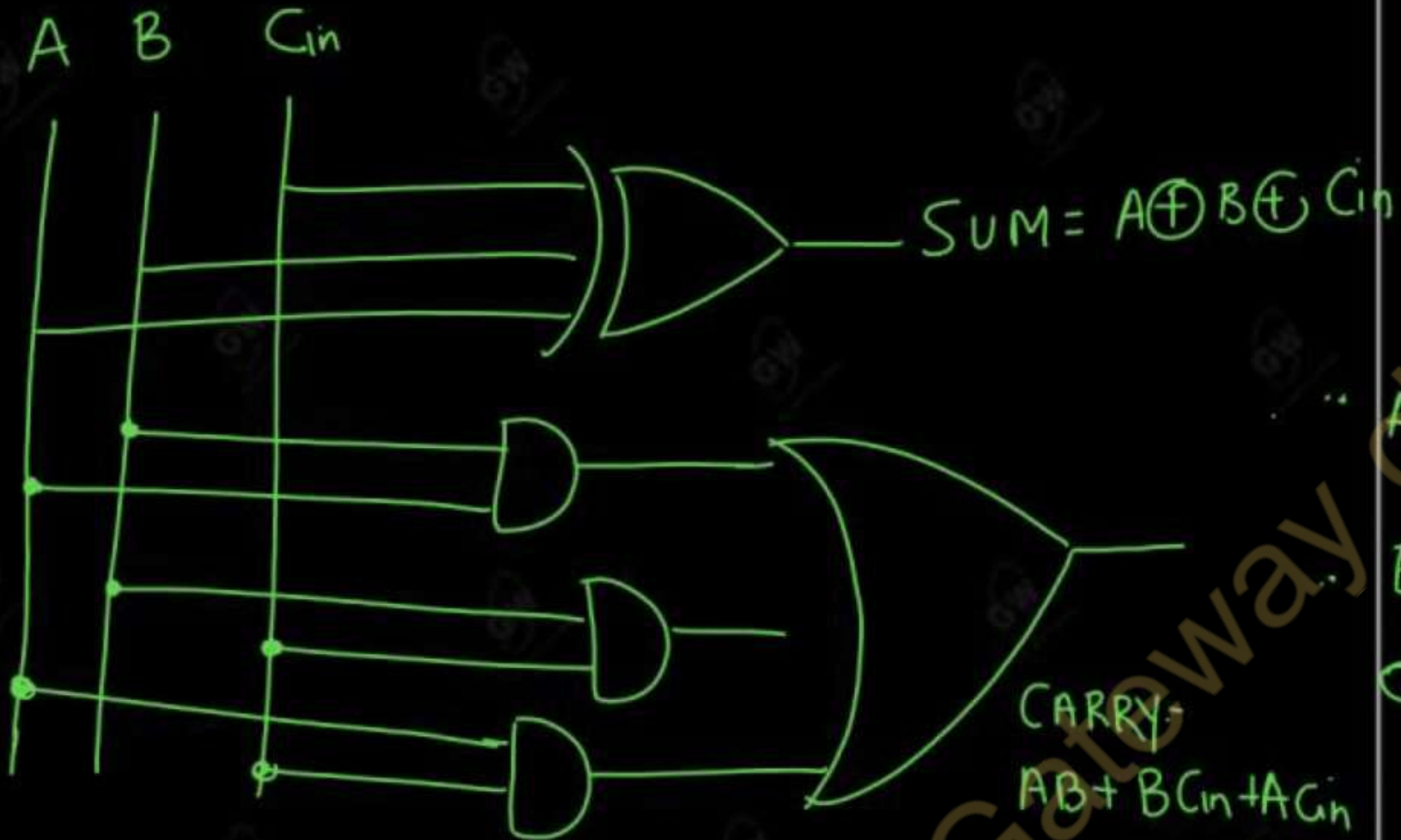
$$\begin{aligned}
 &\bar{A}BC_{in} + ABC_{in} \\
 &BC_{in}(\bar{A} + A) \\
 &BC_{in}
 \end{aligned}$$

Carry =

$$AB + Bc_{in} + AC_{in}$$

$$\begin{aligned}
 &AB C_{in} + \bar{A}\bar{B} C_{in} \\
 &A C_{in} (B + \bar{B}) \\
 &A C_{in}
 \end{aligned}$$

$$\begin{aligned}
 C_{in} &\rightarrow 0 & 0+1 \\
 C_{in} &\rightarrow 1
 \end{aligned}$$



Computer organization and architecture

UNIT 2

Arithmetic and logic unit

Today's Target

- FULL adder using half adder
- Ripple carry adder
- Carry look ahead adder
- AKTU PYQs

By PRAGYA RAJVANSHI

B.Tech, M.Tech(C.S.E.)

XOR Exclusive OR



A	B	$A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

Full adder using half adder

Half adder



FULL Adder

$$\text{SUM} = A \oplus B \oplus C_{in}$$

$$\text{CARRY} = AB + BC_{in} + AC_{in}$$

$$A \rightarrow 1$$

$$A' \rightarrow 0$$

$$ABC_{in} + AB\bar{C}_{in}$$

$$AB(C_{in} + \bar{C}_{in}) = AB$$

$$\bar{A}BC_{in} + ABC_{in}$$

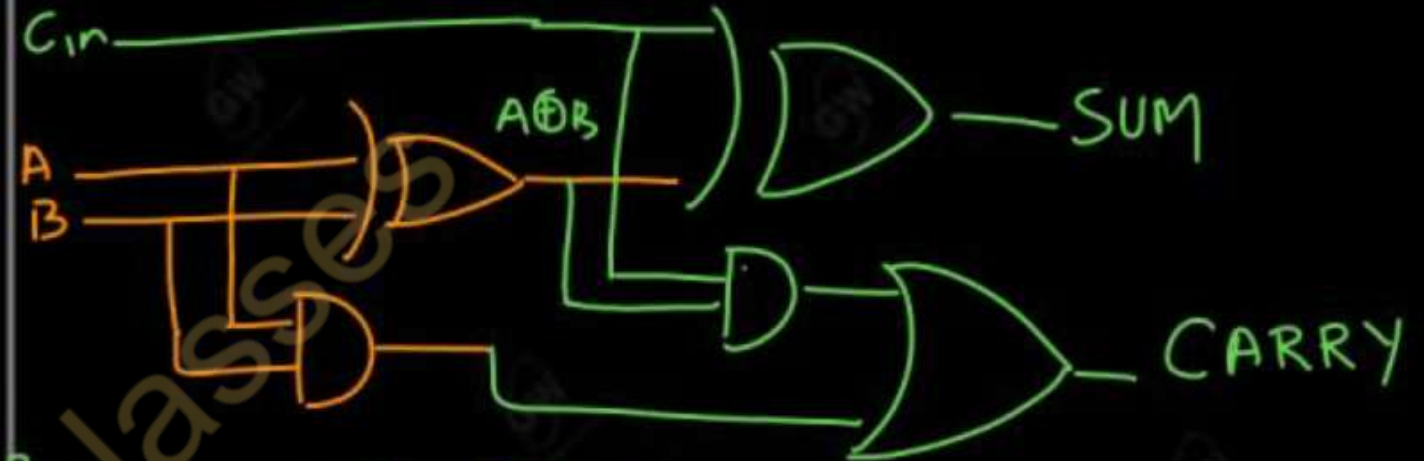
$$BC_{in}(\bar{A} + A) = BC_{in}$$

$$A\bar{B}C_{in} + A\bar{B}\bar{C}_{in}$$

$$A\bar{B}(C_{in} + \bar{C}_{in}) = A\bar{B}$$

$$\text{CARRY} =$$

$$AB + BC_{in} + A\bar{B}$$



$$\text{SUM} = A \oplus B \oplus C_{in}$$

$$\text{CARRY} = (A \oplus B) \cdot C_{in} + AB$$

$$(A\bar{B} + \bar{A}B)C_{in} + AB(C_{in} + \bar{C}_{in})$$

$$A\bar{B}C_{in} + \bar{A}B\bar{C}_{in} + ABC_{in} + AB\bar{C}_{in}$$

	AB	$\bar{A}\bar{B}$	$\bar{A}B$	$A\bar{B}$	AB
$\bar{C}_{in}$	00	01	11	10	
0	0	0	1	1	2
1	1	1	1	1	3

Full adder using half adder

Gateway classes

Ripple carry adder

- It is a **combinational logic circuit**.
- It is **used of adding two n bit binary number**.
- It requires **n full adder** in its circuit for adding **two n bit binary number**.
- It is also know as **n bit parallel adder**.

$$\begin{array}{r}
 C_4 \quad C_3 \quad C_2 \quad C_1 \quad C_0 \text{ — Initial carry} \\
 A_3 \quad A_2 \quad A_1 \quad A_0 \quad A \\
 B_3 \quad B_2 \quad B_1 \quad B_0 \quad B \\
 \hline
 S_3 \quad S_2 \quad S_1 \quad S_0
 \end{array}$$

C_1 depend C_0

C_2 depend C_1

C_3 depend C_2

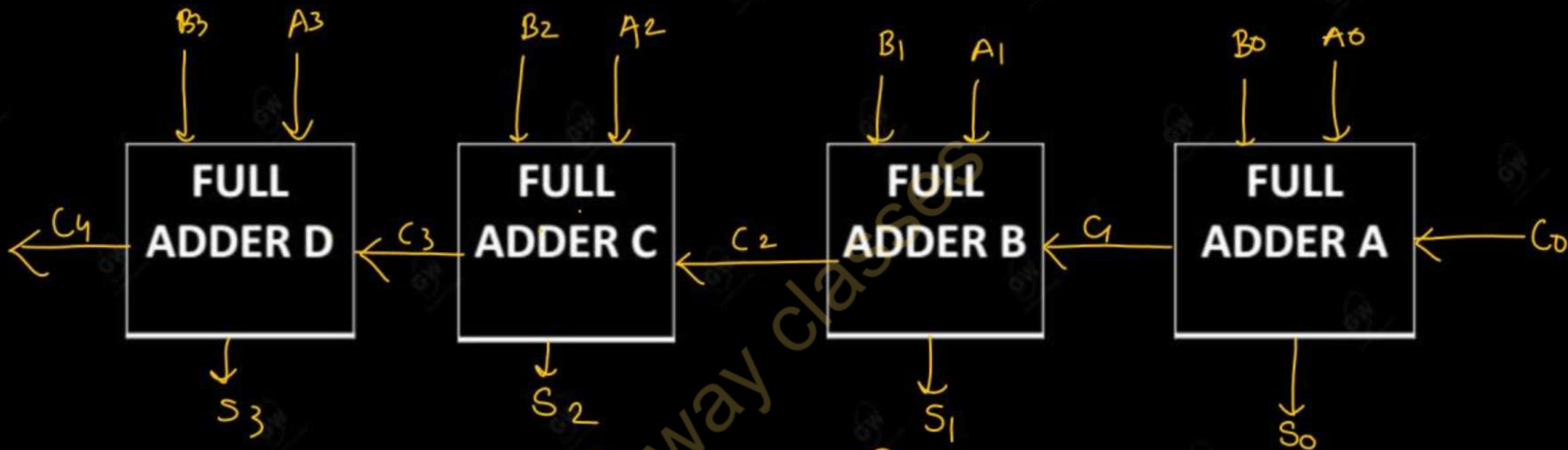
↓

↓

Disadvantages

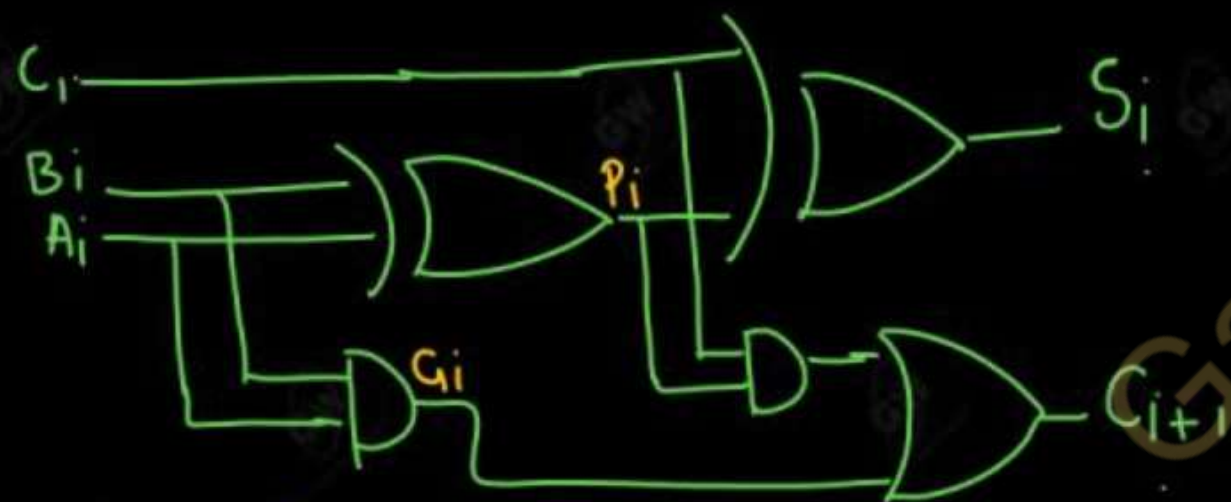
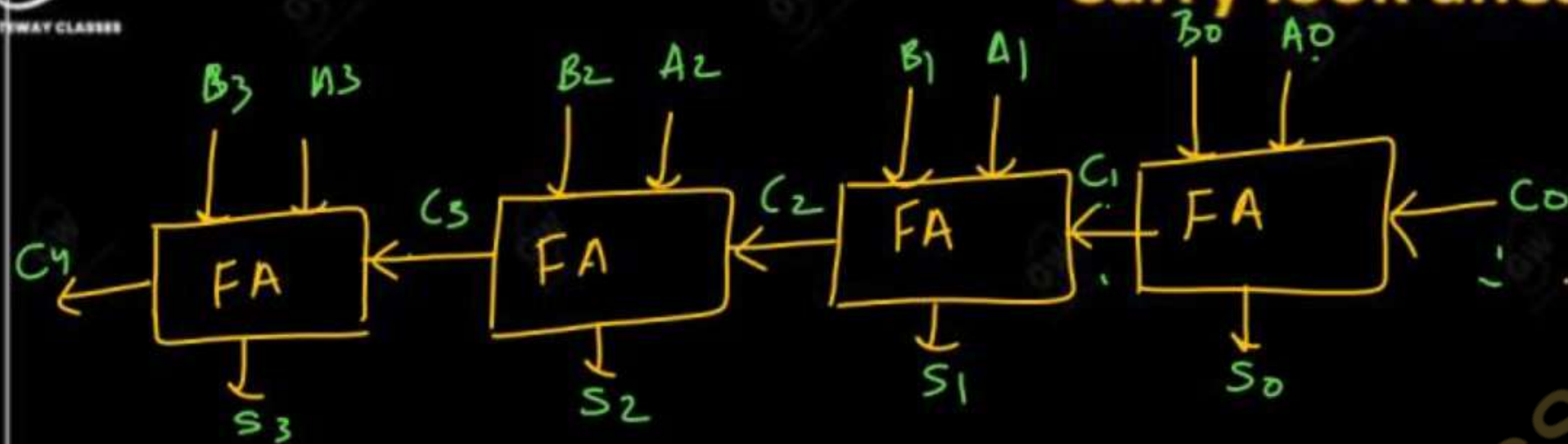
- Each full adder has to wait until the carry bit become available from its adjacent full adder.
- This increase the propagation delay. Due to this, ripple adder becomes extremely slow.

$$\begin{array}{r}
 C_4 \quad C_3 \quad C_2 \quad C_1 \quad C_0 \\
 1 \quad 1 \quad 0 \quad 0 \quad 0 \text{ — Carry} \\
 1 \quad 1 \quad 1 \quad 0 \\
 1 \quad 1 \quad 0 \quad 0 \\
 \hline
 1 \quad 0 \quad 1 \quad 0 \\
 S_3 \quad S_2 \quad S_1 \quad S_0
 \end{array}$$



Propagation Delay

Carry look ahead adder (4-bit)



$$A_i \oplus B_i$$

$$A_i \bar{B}_i + \bar{A}_i B_i$$

$$S_i = P_i \oplus C_i$$

$$C_{i+1} = P_i C_i + G_i$$

$P_i = A_i \oplus B_i$ { carry propagator }

$G_i = A_i B_i$ { carry generator }

A_i	B_i	C_i	S_i	C_{i+1}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

$$S_i = P_i \oplus C_i$$

$$C_{i+1} = P_i C_i + G_i$$

$$S_i = P_i \oplus C_i$$

$$i=0 \quad S_0 = P_0 \oplus C_0$$

$$i=1 \quad S_1 = P_1 \oplus C_1$$

$$i=2 \quad S_2 = P_2 \oplus C_2$$

$$C_{i+1} = P_i C_i + G_i$$

$$C_1 = P_0 C_0 + G_0$$

$$C_2 = P_1 C_1 + G_1$$

$$C_3 = P_2 C_2 + G_2$$

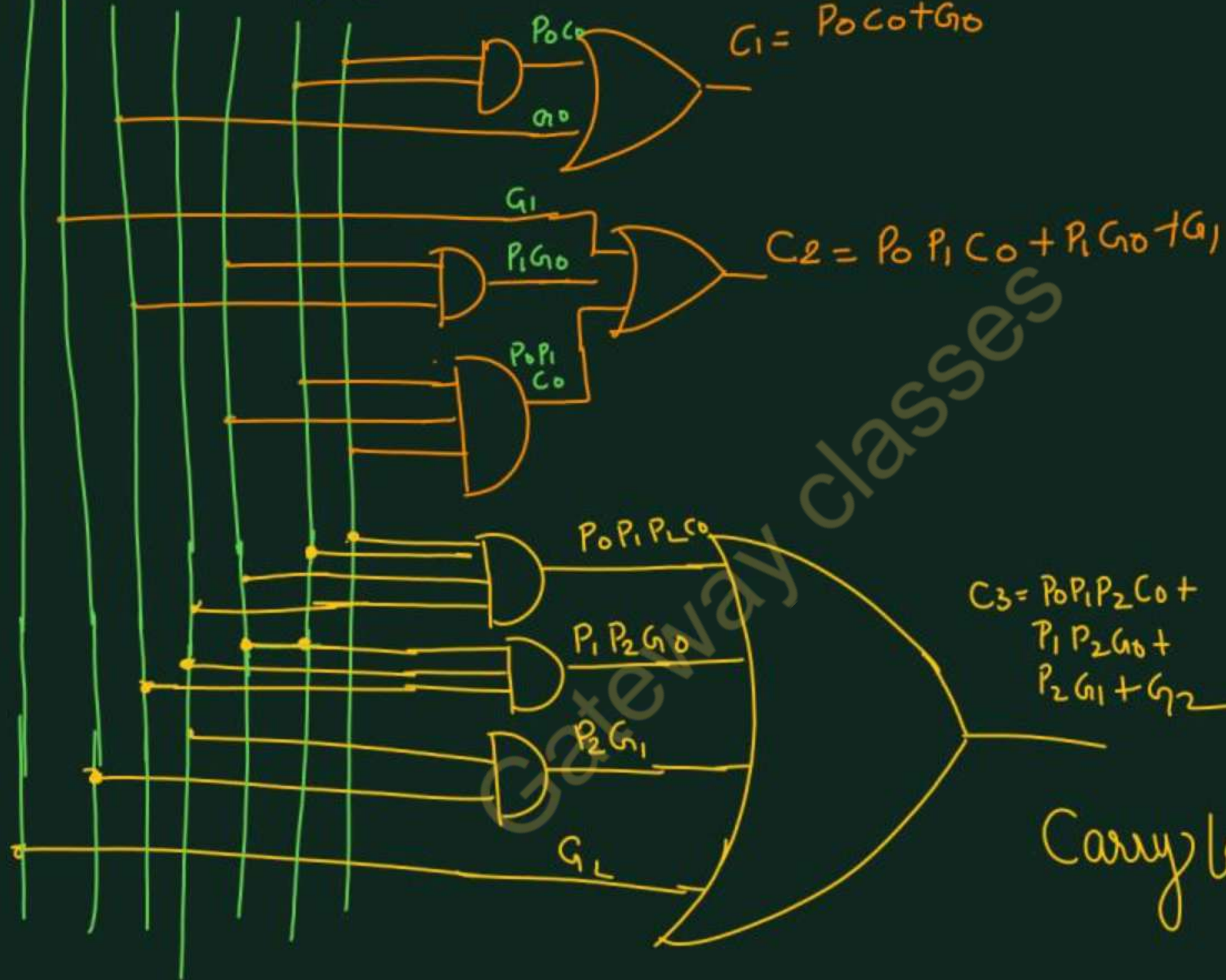
$$C_2 = P_1 (P_0 C_0 + G_0) + G_1$$

$$C_2 = P_0 P_1 C_0 + P_1 G_0 + G_1$$

$$C_3 = P_2 (P_0 P_1 C_0 + P_1 G_0 + G_1) + G_2$$

$$C_3 = P_0 P_1 P_2 C_0 + P_1 P_2 G_0 + P_2 G_1 + G_2$$

C_2 C_1 C_0 P_2 P_1 P_0 C_0

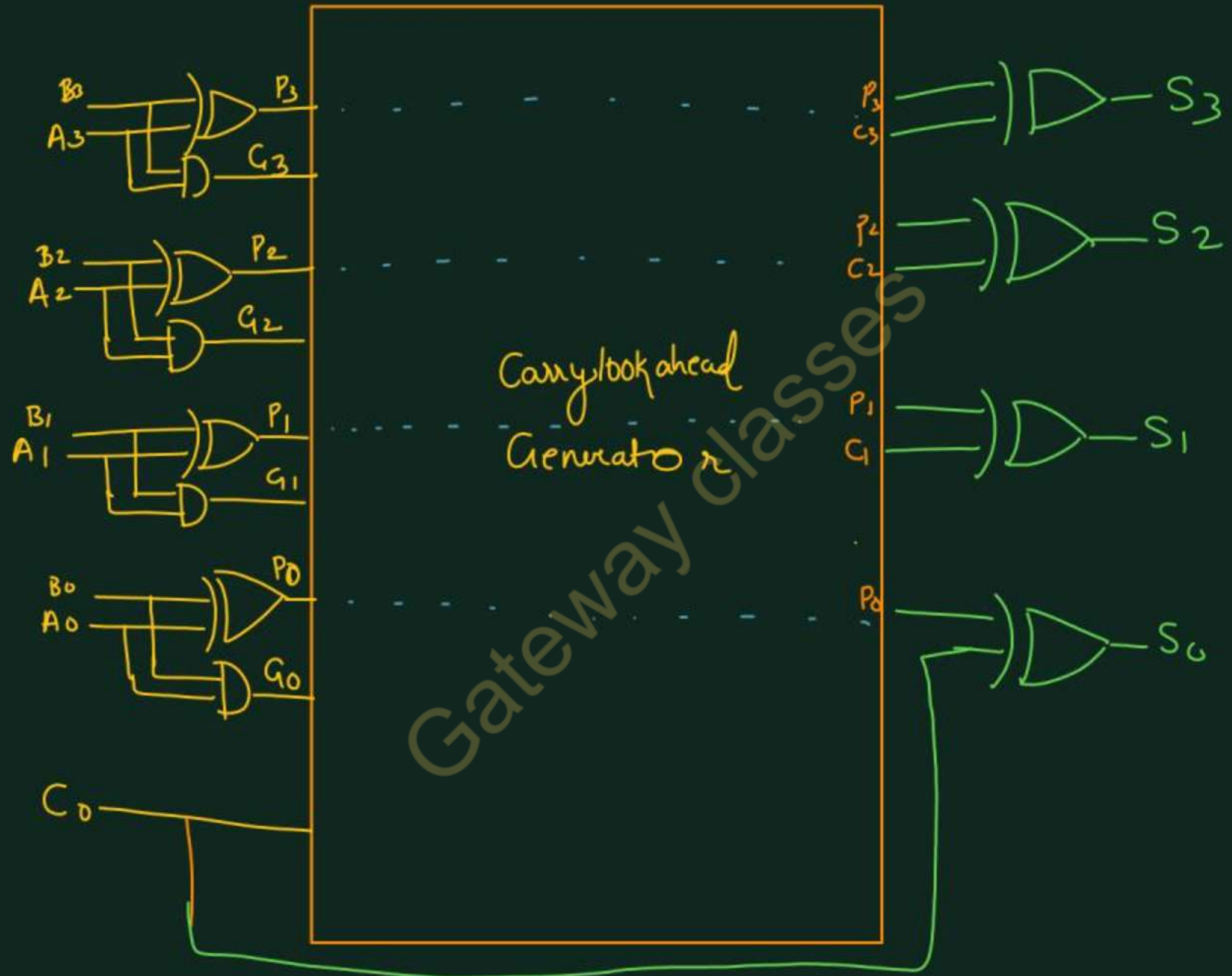


$$C_1 = P_0C_0 + G_0$$

$$C_2 = P_0P_1C_0 + P_1G_0 + G_1$$

$$C_3 = P_0P_1P_2C_0 + P_1P_2G_0 + P_2G_1 + G_2$$

Carry look ahead Generator



4-bit
Carry look
ahead
adder.

Q.1	Explain 5 bit parallel adder. Or Explain 4 bit parallel adder.	AKTU 2015-26 AKTU 2016-17
Q.2	Draw the block diagram of full adder using half adder.	<u>AKTU 2017-18</u>
Q.3	Explain 4 bit/3 bit carry look ahead adder	AKTU 2023-24 AKTU 2022-23 AKTU 2021-22 AKTU 2020-21

Computer organization and architecture

UNIT 2

Arithmetic and logic unit

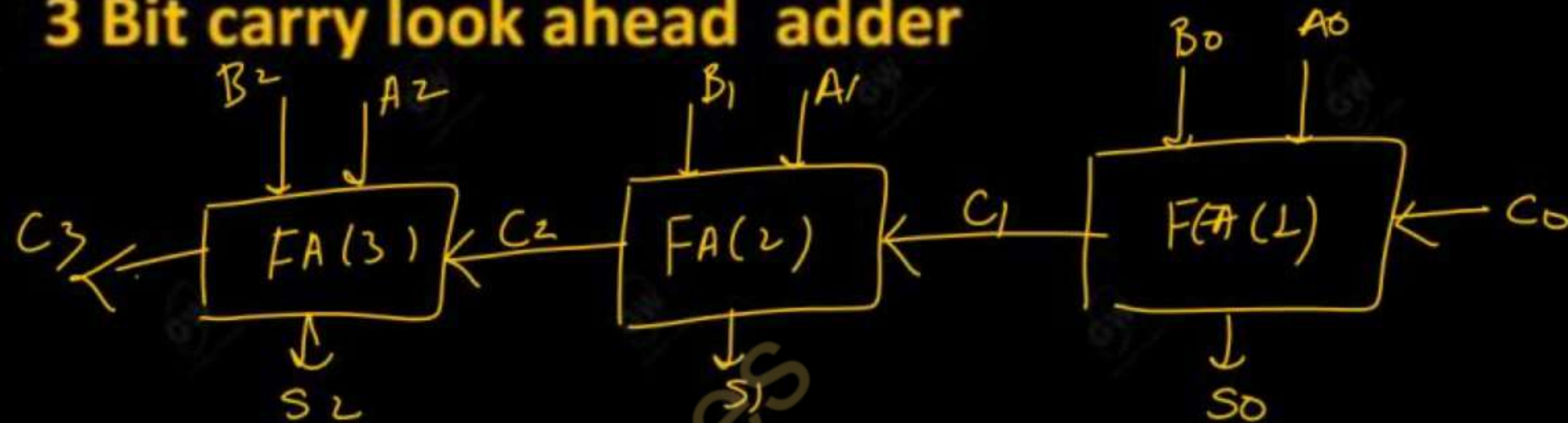
Today's Target

- 3 bit Carry look ahead adder
- ALU PQs

By PRAGYA RAJVANSHI

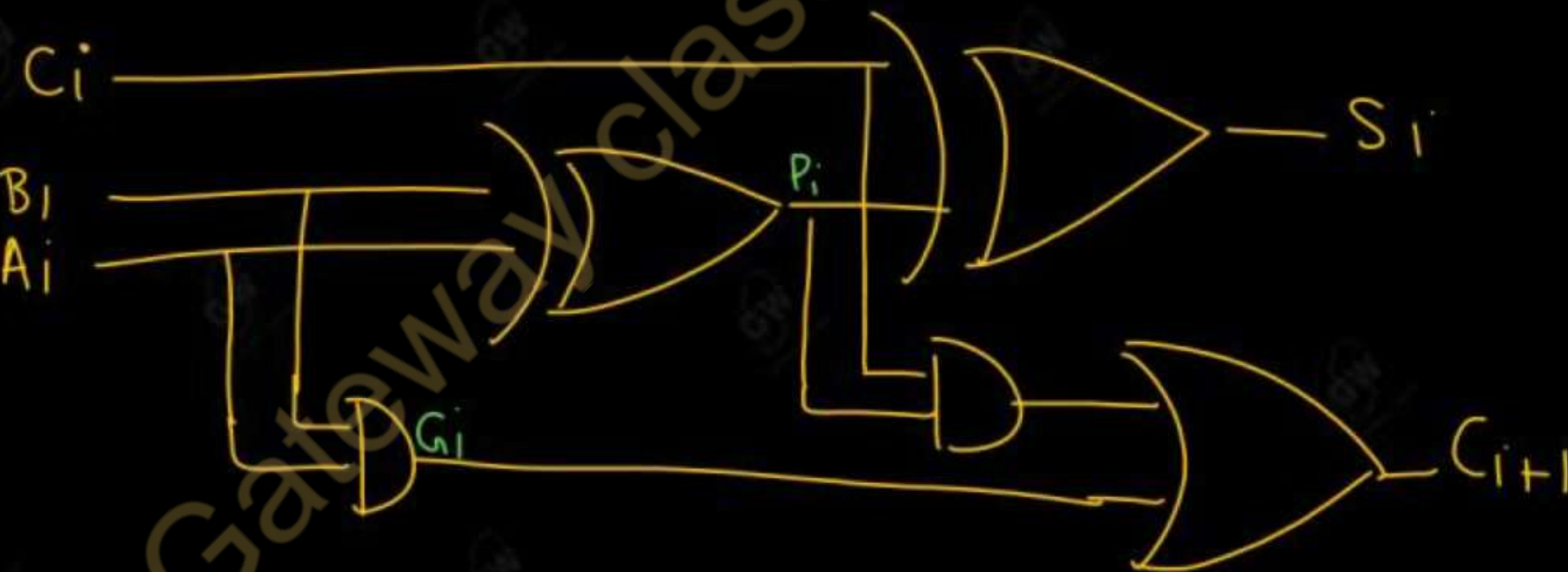
B.Tech, M.Tech(C.S.E.)

3 Bit carry look ahead adder



C_3	C_2	C_1	C_0	
A_2	A_1	A_0	A	
B_2	B_1	B_0	B	
S_2	S_1	S_0		

P_i (Carry propagator) = $A_i \oplus B_i$
 G_i (Carry generator) = $A_i B_i$



3 Bit carry look ahead adder

A_i	B_i	C_i	S_i	C_{i+1}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

$$P_i = A_i \oplus B_i$$

$$G_i = A_i B_i$$

$$\bar{A}_i B_i + A_i \bar{B}_i$$

$$S_i = P_i \oplus C_i$$

$$C_{i+1} = P_i C_i + G_i$$

$$i=0 \quad S_0 = P_0 \oplus C_0$$

$$i=0$$

$$C_1 = P_0 C_0 + G_0$$

$$i=L \quad S_1 = P_1 \oplus C_1$$

$$i=L$$

$$i=2 \quad S_2 = P_2 \oplus C_2$$

$$C_2 = P_1 C_1 + G_1$$

$$P_1 (P_0 C_0 + G_0) + G_1$$

$$C_2 = P_0 P_1 C_0 + P_1 G_0 + G_1$$

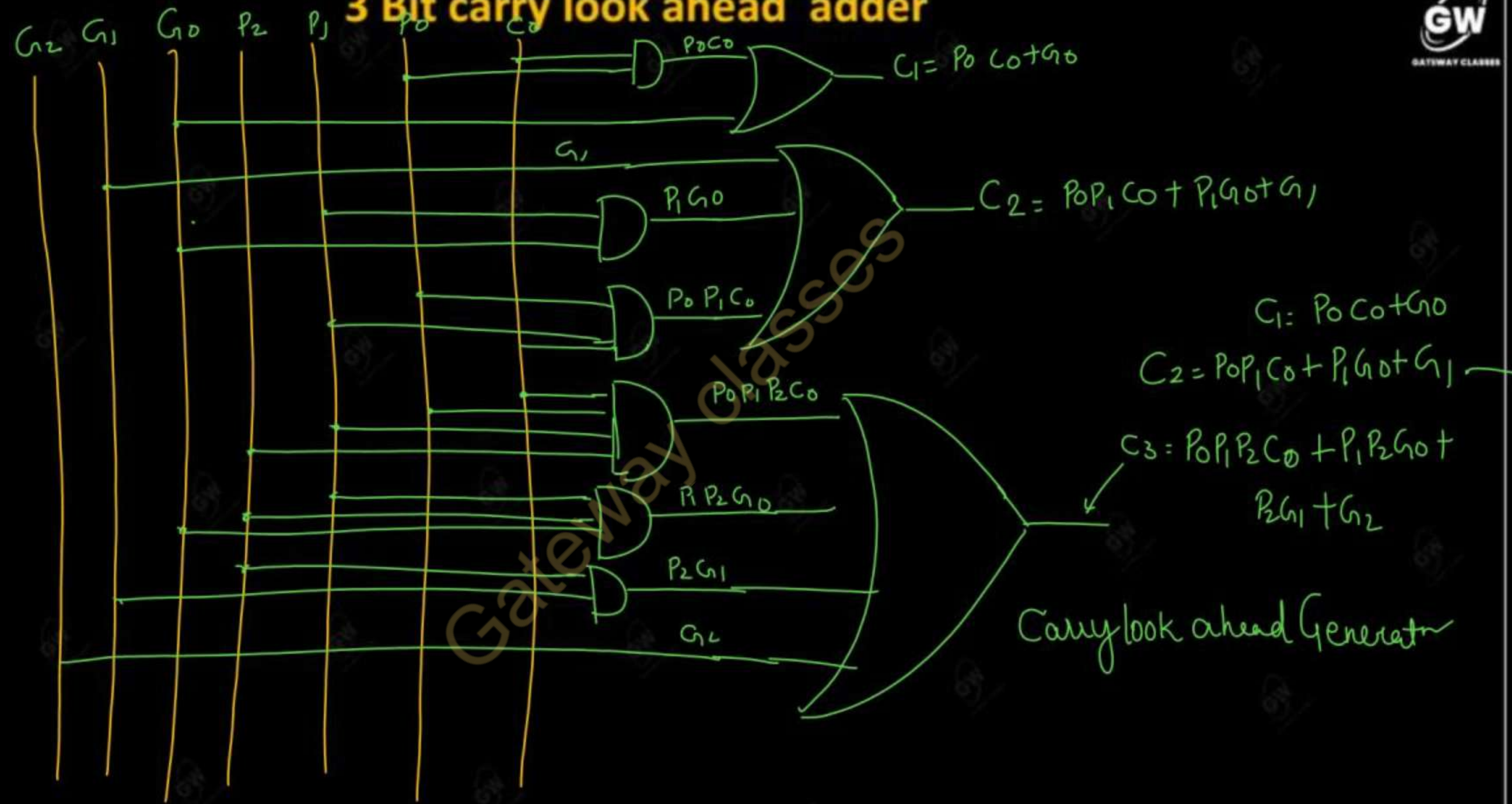
$$i=2$$

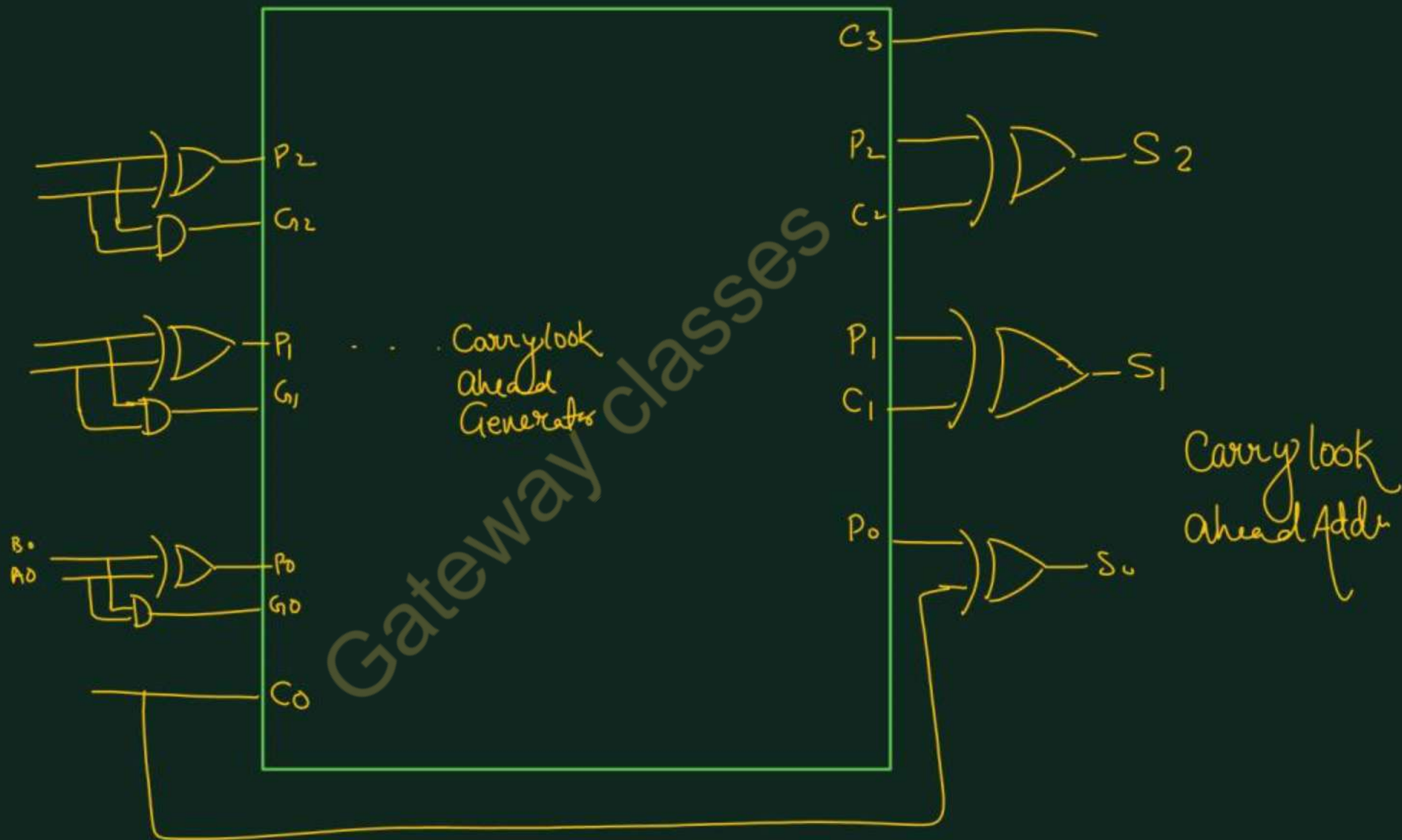
$$C_3 = P_2 C_2 + G_2$$

$$P_2 (P_0 P_1 C_0 + P_1 G_0 + G_1) + G_2$$

$$C_3 = P_0 P_1 P_2 C_0 + P_1 P_2 G_0 + P_2 G_1 + G_2$$

3 Bit carry look ahead adder



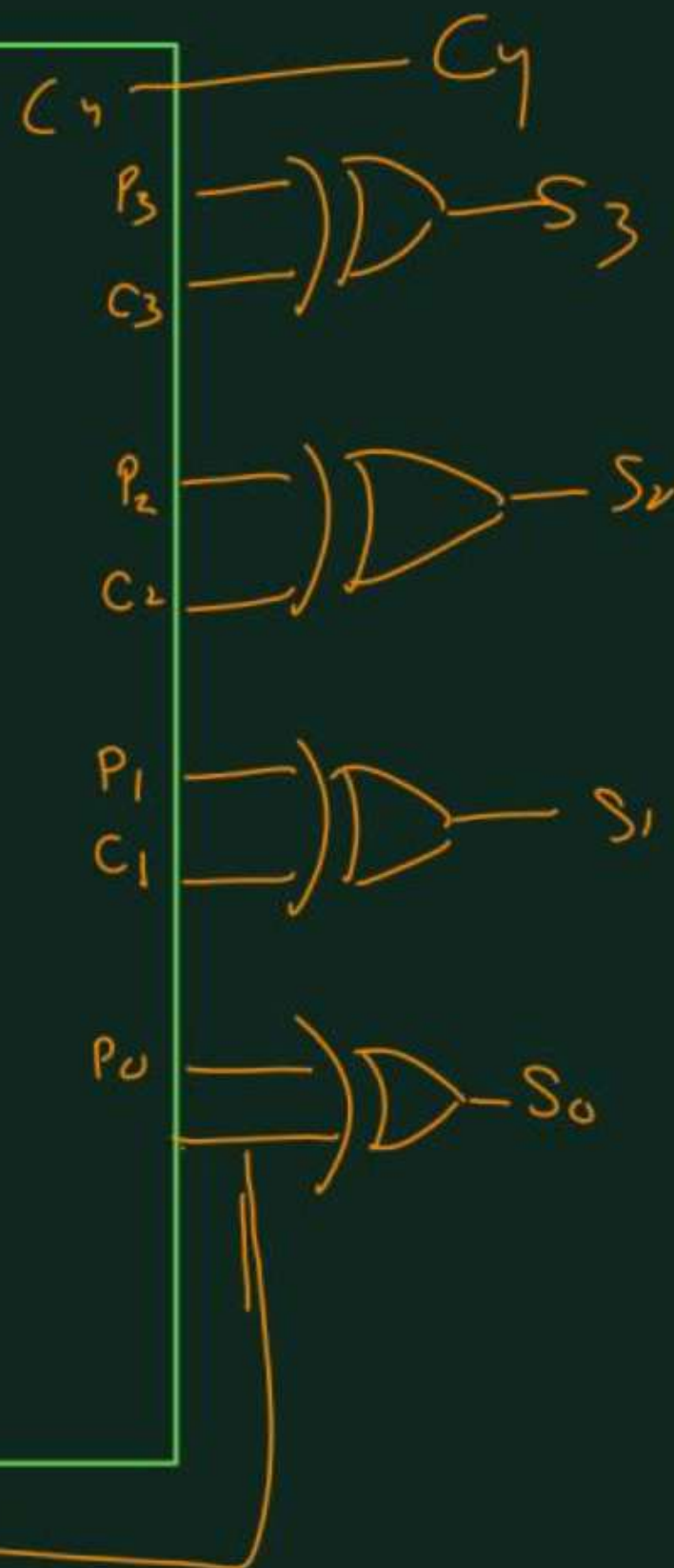


C_7	C_3	C_2	C_1	C_0
A_7	A_4	A_1	A_0	
B_7	B_4	B_1	B_0	
S_3	S_2	S_1	S_0	

Carry look
ahead Adder



4 bit
Carry look
ahead Gen



Thank
you